INVOICEFLOW: FULL-STACK ACCOUNTS PAYABLE AND FINANCIAL WORKFLOW MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

SAI SRINIVAS M.D. (310624243153)

in partial fulfillment for the award of the Degree

of

BACHELOR OF TECHNOLOGY

In

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



EASWARI ENGINEERING COLLEGE

(AN AUTONOMOUS INSTITUTION)
AFFILIATED TO ANNA UNIVERSITY,
CHENNAI

OCTOBER 2026

EASWARI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this project report titled "**INVOICEFLOW: FULL-STACK ACCOUNTS PAYABLE AND FINANCIAL WORKFLOW MANAGEMENT SYSTEM**" is the Bonafide work of "**SAI SRINIVAS M.D. (310624243153)**", who carried out the project work under my supervision.

HEAD OF THE DEPARTMENT

Dr. T. Kalai Selvi

Easwari Engineering College
Ramapuram, Chennai – 600089.

SUPERVISOR

Ms. D. S. K. Harshitha

Easwari Engineering College
Ramapuram, Chennai – 600089.

EASWARI ENGINEERING COLLEGE

Certificate of Viva-voce Examination

This is to certify that the Viva Voce Examination of the Project Work entitled "INVOICEFLOW: FULL-STACK ACCOUNTS PAYABLE AND FINANCIAL WORKFLOW MANAGEMENT SYSTEM" submitted by SAI SRINIVAS M.D. (Reg. No. 310624243153) in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY was conducted on 28-10-2026 in the Department of Artificial Intelligence and Data Science, Easwari Engineering College, Ramapuram – Chennai.

Internal Examiner

Date: 28/10/26

ABSTRACT

Accounts payable (AP) is the process through which an organisation receives supplier invoices, verifies them, obtains approval, pays them and proves afterwards that the payment reached the intended supplier. When it is run through e-mail and spreadsheets, the usual results are duplicate payments, invoices approved by the person who entered them, untraceable changes to vendor bank details and a reconciliation done by hand. InvoiceFlow is a full-stack web application that turns this process into an explicit, permission-controlled workflow covering invoice capture, vendor management, rule-driven approvals, payment-run preparation, bank-statement reconciliation, reporting and a tamper-evident audit log.

The client is a React 18 and TypeScript application built with Vite, and the server uses Node.js, Express, TypeScript, MongoDB and Mongoose, with Redis and BullMQ for background jobs and Docker Compose for development. Business rules are kept in separate domain functions: an invoice state machine, segregation-of-duties checks, an approval-matrix engine and a reconciliation matching engine. Money is held as integer paise, vendor bank numbers are encrypted with AES-256-GCM, access is decided by 39 permissions, and every change is written to a hash-chained audit log.

InvoiceFlow does not transfer money. An authorised user generates a payment CSV file, and an invoice becomes paid only when a bank statement line is matched or a person records the bank reference. This report separates implemented features, automated tests present in the repository, and observed behaviour. The 74 tests reported by the author were not re-run for this report, and real MongoDB transactions, Redis processing, e-mail delivery, Google sign-in and OCR remain unverified.

ACKNOWLEDGEMENT

I express my sincere gratitude to the Principal Dr. P. Deiva Sundari and the Management of our college for providing me with the opportunity to carry out this project work.

I convey my heartfelt thanks to the Head of the Department Dr. T. Kalai Selvi, Department of Artificial Intelligence and Data Science, for the continuous encouragement and support.

I am extremely thankful to my Project Guide Ms. D. S. K. Harshitha, Department of Artificial Intelligence and Data Science, for valuable guidance, suggestions, and motivation throughout the completion of this project.

I would also like to extend my gratitude to all the faculty members of the Department of Artificial Intelligence and Data Science for their support.

Finally, I thank my parents and friends for their constant encouragement and help during the course of this project.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE No.
	ABSTRACT	**i**
	ACKNOWLEDGEMENT	**ii**
	LIST OF TABLES	**v**
	LIST OF FIGURES	**vi**
1.	**INTRODUCTION**	**1**
	1.1 OVERVIEW	1
	1.2 PROBLEM DEFINITION	1
	1.3 OBJECTIVE	2
	1.4 SCOPE OF THE PROJECT	2
	1.5 MOTIVATION	3
2.	**LITERATURE SURVEY**	**4**
	2.1 LITERATURE REVIEW	4
	2.1.1 Integrity controls and separation of duty	4
	2.1.2 Role-based access control	4
	2.1.3 Tamper-evident audit logging	5
	2.1.4 Web application architecture and technical background	5
	2.2 EXISTING SYSTEM	5
	2.3 PROPOSED SYSTEM	6
3.	**SYSTEM ANALYSIS**	**7**
	3.1 SYSTEM REQUIREMENTS	7
	3.1.1 Functional requirements	7
	3.1.2 Non-functional requirements	8
	3.1.3 User roles and permissions	8
	3.2 SOFTWARE AND HARDWARE REQUIREMENTS	9
4.	**METHODOLOGY**	**10**
	4.1 SYSTEM ARCHITECTURE	10
	4.2 MODULE DESCRIPTION	12
	4.3 CLASS DIAGRAM	13
	4.4 SEQUENCE DIAGRAM	15
5.	**RESULTS AND DISCUSSION**	**17**
	5.1 IMPLEMENTATION OVERVIEW	17
	5.2 KEY FUNCTIONS (CODE EXPLANATION)	18

5.2.1 Invoice state machine 18
5.2.2 Segregation-of-duties policy 19
5.2.3 Choosing approvers 20
5.2.4 Payment-run release and CSV generation 21
5.2.5 Reconciliation matching 22
5.2.6 Hash-chained audit log 23
5.2.7 Authentication and permission middleware 24
5.2.8 Transactions and money handling 24
5.3 TESTING AND VERIFICATION STATUS 25
6. CONCLUSION & FUTURE WORK 27
6.1 CONCLUSION 27
6.2 FUTURE ENHANCEMENTS 27
APPENDIX 29
SOURCE CODE 29
SCREENSHOTS OF THE PROJECT 34
REFERENCES 42

LIST OF TABLES

TABLE NO.	TABLE TITLE	PAGE No.
2.1	Typical weaknesses of a manual AP process and the InvoiceFlow response	5
3.1	Functional requirements	7
3.2	Role templates (from permissions.ts and README)	8
3.3	Software stack (from package.json, docker-compose.yml and CI configuration)	9
3.4	Recommended development environment (recommendation, not specified by the repository)	9
4.1	Principal modules and their implementation files	12
5.1	Testing and verification status	25
A.1	Configuration variables	30
A.2	Security mechanisms present in the source	32
A.3	Selected API routes (prefix /api) and required permissions	33

LIST OF FIGURES

FIGURE NO.	FIGURE TITLE	PAGE No.
4.1	Three-tier architecture of InvoiceFlow, derived from the source layout and docker-compose.yml	11
4.2	Invoice status transitions defined in domain/stateMachine.ts	13
4.3	Entity relationships: organisation, users, roles, delegation and vendors	14
4.4	Entity relationships: invoices, intake documents, approval rules and approval steps	14
4.5	Entity relationships: payment runs, payments and bank reconciliation (AuditLog is append-only and references entities by type and id)	15
4.6	Sequence diagram of the invoice-to-reconciliation workflow	16
A.1	Sign-in page (entry point of the application)	34
A.2	Dashboard: status mix, key figures and weekly cash-need chart	34
A.3	Invoice INV-2291 in review with two sequential approval steps	35
A.4	Approver's queue showing invoices waiting on the user and the approval limit check	35
A.5	Draft payment run: the preparer is told that someone else must release it	36
A.6	Released payment run with bank-file download; payments are Released, not Paid	36
A.7	Bank statement import preview with the selected layout and parsed lines	37
A.8	Statement lines needing attention: an exception and an unmatched debit	37
A.9	Statement lines matched to payments by reference	38
A.10	Users and roles: people with approval limits	38
A.11	Roles and permissions with editable role templates	39
A.12	Audit log with the chain-verified confirmation	39
A.13	Vendors: status filters with counts and the bank-change approval queue	40
A.14	Reports: AP aging chart, overdue buckets in red	40

A.15	Reports: vendor spend ranking	41
A.16	Public landing page shown before sign-in (static content, no sample data)	41

CHAPTER 1
INTRODUCTION

This chapter introduces the background, problem definition, objectives, scope and motivation of the InvoiceFlow system.

1.1 OVERVIEW

InvoiceFlow is a web-based accounts-payable and financial-workflow management system. It supports the path that a supplier invoice follows inside an organisation: it is captured, coded and checked; approved by the right people within their authority; scheduled into a payment run; released by a second person; paid by the bank outside the system; and finally matched against the bank statement and recorded in an audit trail. The system is built as a full-stack TypeScript application: a React 18 single-page client, an Express REST API, MongoDB for persistence, and Redis with BullMQ for background work, packaged for development with Docker Compose.

The project is multi-tenant by construction (every record carries an organisation identifier and a query without one is rejected), currency handling is in Indian rupees with Goods and Services Tax (GST) and Tax Deducted at Source (TDS) fields, and the validation routines understand Indian identifiers such as GSTIN, PAN and IFSC. Six role templates (Requester, Accounts Team, Approver, Finance Head, Admin, Auditor) are provided and can be edited, because the backend authorises on permissions and never on role names.

1.2 PROBLEM DEFINITION

Manual accounts-payable handling creates a recurring set of financial and operational problems:

- **Duplicate and erroneous payments** when the same invoice arrives twice or an amount is keyed incorrectly.
- **Weak segregation of duties** when the person who enters or edits an invoice can also approve it, or the person who prepares a payment batch can also release it.
- **Vendor-fraud exposure** when bank details change without independent verification.
- **Approval delays and opacity**, because approvers, limits and escalation depend on e-mail threads rather than rules.

- **Manual reconciliation**, where bank-statement lines are compared with payments by eye and an invoice is assumed to be paid once a transfer has been requested.
- **Poor auditability**, because spreadsheet and e-mail histories can be edited and do not prove who did what and when.

The problem addressed by this project is to provide a single, controlled system in which each of these steps is explicit, permission-checked, validated, and recorded.

1.3 OBJECTIVE

The objectives of the project are:

1. To model the invoice lifecycle as a closed state machine in which an invoice can only move through legal transitions.
2. To validate financial data at entry: integer-paise money, GST/PAN/IFSC formats, tax arithmetic, TDS taken from organisation settings, and duplicate-invoice detection.
3. To drive approvals from configurable rules (amount range, department, cost centre, vendor category) while ensuring every approver's limit covers the whole invoice and no one approves their own work.
4. To prepare payment runs with maker-checker release, optional dual signature above a threshold, and generation of a payment CSV file, without claiming that money has moved.
5. To import bank statements through configurable column mappings and reconcile them against released payments with an explainable matching engine.
6. To provide dashboards, operational reports and an append-only, hash-chained audit log, with role-based access control throughout.

1.4 SCOPE OF THE PROJECT

Intended users: requesters who raise bills, accounts staff who capture invoices and prepare payment runs, approvers, a finance head who releases payments and approves vendor changes, an administrator who manages users and roles, and an auditor with read-only access.

In scope (implemented in the source): invoice capture with PDF text extraction and manual verification; vendor proposal, approval and bank-change approval; approval rules, sequential or parallel steps, delegation and escalation;

payment runs, payment holds, generic CSV generation, manual payment confirmation; bank-account and statement-profile configuration, statement import, matching and manual match approval; dashboard and eight report endpoints; audit log with chain verification; invitation-only user management.

Out of scope / not implemented: real bank transfers or payout-provider APIs (the provider interface deliberately returns "not configured"), bank-specific payment-file formats, built-in tax rates, bank-specific statement layouts, multi-factor authentication (fields exist, flow does not), purchase-order and goods-receipt matching, accounting-software integration beyond a generic journal CSV, and languages other than English. These are treated as limitations and future work in Chapter 6.

1.5 MOTIVATION

Accounts payable is a high-trust, high-consequence process: errors are expensive and fraud commonly exploits missing controls rather than technical weaknesses. Classical security literature already identifies separation of duty and well-formed transactions as the core of commercial integrity [4], and access-control research provides role-based models for assigning authority [6], [7]. The motivation for InvoiceFlow was to apply these principles in a practical application that a small finance team could actually operate, and to do so honestly: the system makes it explicit that a payment file is not a payment, and that "paid" must be confirmed by evidence from the bank. The project also provided an opportunity to combine web-application engineering, transactional data modelling, security engineering and data-matching algorithms in one coherent system.

CHAPTER 2
LITERATURE SURVEY

This chapter reviews the literature on which the project is built, the way accounts payable is handled today, and the position that InvoiceFlow takes.

2.1 LITERATURE REVIEW

This survey separates **published research and standards** from **general technical background**. No survey of commercial AP-automation vendors or of academic studies measuring AP automation outcomes was carried out for this report; such a comparison is left as **[OPTIONAL: add vendor/industry survey if required by the department]**.

2.1.1 Integrity controls and separation of duty (published research)

Clark and Wilson argued that commercial systems need integrity policies distinct from military confidentiality models, built around well-formed transactions and separation of duty, so that no single person can alter data in ways that defeat the intent of the control [4]. InvoiceFlow applies the same idea concretely: invoice state changes only through named actions, and the policy module forbids self-approval, preparer-releases-own-run, proposer-approves-own-vendor-change, and one person giving both release signatures. Saltzer and Schroeder's design principles, including least privilege, fail-safe defaults and complete mediation, motivated the decision to load permissions from the database on every request and to deny by default [5].

2.1.2 Role-based access control (published research and standard)

Sandhu et al. introduced a family of role-based access-control models [6], and Ferraiolo et al. proposed the NIST RBAC standard that formalised roles, permissions and constraints such as separation of duty [7]. InvoiceFlow follows the central idea, assigning permissions to roles and roles to users, with one deliberate variation: roles are editable data, and the API checks permissions, so that a changed role takes effect immediately.

2.1.3 Tamper-evident audit logging (published research)

Schneier and Kelsey described secure audit logs whose entries are cryptographically linked so that later alteration or deletion can be detected [8]. InvoiceFlow's audit log uses a simpler hash chain, in which each entry stores the hash of its predecessor, and provides a verification endpoint. It does not use the forward-secure key evolution of [8] and should not be presented as equivalent to it.

2.1.4 Web application architecture and technical background (documentation, not research)

The REST style described by Fielding [2] underlies the API, and the separation between domain logic and persistence follows widely taught enterprise-application patterns, including the Money and Service Layer patterns described by Fowler [1]. Storing money in integer minor units follows that Money pattern and avoids floating-point error. JSON Web Tokens are specified in RFC 7519 [3]; bcrypt originates from Provos and Mazières [9]; AES in Galois/Counter Mode is specified in NIST SP 800-38D [10]. Multi-document ACID transactions in MongoDB require a replica set [11]. Payment files are exchanged as CSV, whose common format is documented in RFC 4180 [12], and spreadsheet "formula injection" through CSV is a documented risk [13]. Security verification guidance for web applications is collected in the OWASP ASVS [14]. Official documentation for the implementation technologies is listed in [15]–[19].

2.2 EXISTING SYSTEM

In many small and mid-sized organisations the existing AP process is a combination of e-mail, shared drives, spreadsheets and bank portals. Typical characteristics are summarised in Table 2.1. These characteristics are general observations about manual processes, not measurements from a particular organisation.

Table 2.1: Typical weaknesses of a manual AP process and the InvoiceFlow response

Aspect	Manual / spreadsheet process	InvoiceFlow design response
Duplicate detection	Relies on memory or manual search	Unique index on vendor + normalised invoice number; file-hash check; override needs a recorded reason
Approval	E-mail chain; limits informal	Rule-matched steps; approver limit must cover full amount; delegation and escalation recorded
Segregation of duties	Convention only	Enforced in the backend policy functions; configurable

Aspect	Manual / spreadsheet process	InvoiceFlow design response
Payment	Bank portal keyed by hand	Payment run, second-person release, hashed CSV; no payout claimed
Reconciliation	Visual comparison	Parsed statements, balance check, ordered matching rules, exception queue
Audit	Editable files	Append-only, hash-chained log with verify endpoint

2.3 PROPOSED SYSTEM

The proposed system is the InvoiceFlow application described in the remaining chapters. Its distinguishing design decisions are: (a) a closed invoice state machine; (b) all policy rules as pure functions separated from I/O; (c) every financial mutation wrapped in a database transaction that also writes its audit entry; (d) a strict distinction between **released** (file generated) and **paid** (bank-confirmed); (e) human verification of every extracted invoice field, with nothing auto-submitted; and (f) explicit statements, in code and documentation, of what is not implemented. The system is not a replacement for a bank integration or an accounting ledger; it is a control layer that prepares, evidences and reconciles payments.

CHAPTER 3

SYSTEM ANALYSIS

This chapter presents the functional and non-functional requirements of the system and the software and hardware needed to run it.

3.1 SYSTEM REQUIREMENTS

3.1.1 Functional requirements

The functional requirements below were derived from the implemented routes, services and pages. Each is supported by source code; Section 5.3 states separately which are covered by automated tests or observed in screenshots.

Table 3.1: Functional requirements

ID	Module	Requirement
FR1	Authentication	Invitation-only accounts; password sign-in; refresh-token rotation; optional Google sign-in for existing users; password change; forced logout.
FR2	Users and roles	Create/edit/deactivate users; define roles as sets of permissions; approval limits and department/cost-centre scope; approval delegation with dates.
FR3	Vendors	Propose vendors (pending until approved); validate GSTIN, PAN, IFSC; store bank account encrypted and masked; bank changes need a second approver; audited reveal.
FR4	Invoice intake	Upload PDF/PNG/JPEG validated by file signature; extract text from PDFs; parse and validate fields; reviewer converts to a draft invoice.
FR5	Invoices	Create/edit draft invoices in paise; tax totals must add up; TDS from organisation settings; duplicate check; comments and queries; attach file.
FR6	Approvals	Match an approval rule; choose eligible approvers; sequential or parallel steps; approve, reject (reason), hold (query), resume, reopen; escalation of overdue steps.
FR7	Payment runs	Add only approved invoices of active vendors with bank details; draft run with reference numbers; hold individual payments; cancel; release with dual signature above threshold; generate and download CSV.
FR8	Payment confirmation	Mark released payments paid with a bank reference, or failed with a reason (invoice returns to approved).
FR9	Reconciliation	Bank accounts; statement mapping profiles with preview; import CSV/XLSX; running-balance check; duplicate refusal; automatic matching; suggestions, exceptions, manual match with second approver, unmatch, classify.
FR10	Reporting	Dashboard, aging, vendor spend, approval turnaround, payments due, cash-flow forecast, TDS summary, reconciliation status, journal export; CSV export for selected reports.

ID	Module	Requirement
FR11	Audit	Append-only audit entries for actions; chain verification; filtered view and export.
FR12	Settings	Segregation-of-duties switches and thresholds, TDS sections, reconciliation tolerance, date window and classifiers.

3.1.2 Non-functional requirements

- **Security:** bcrypt password hashing, account lockout, 15-minute in-memory access tokens, rotating httpOnly refresh cookies with reuse detection, helmet, CORS allow-list, rate limiting, AES-256-GCM for bank account numbers, file-signature validation, authenticated file download.
- **Data integrity:** integer-paise money, zod request validation, unique indexes, multi-document transactions for financial mutations (require a MongoDB replica set), idempotent release, and a recorded SHA-256 hash of each generated payment file that is re-checked on download.
- **Auditability:** every significant action creates a chained audit entry within the same transaction.
- **Maintainability:** domain rules are pure functions with unit tests; routes are thin; types are shared through TypeScript.
- **Deployability:** Docker Compose for development; example production Compose file, Caddy configuration and nginx configuration are included but are examples only.

3.1.3 User roles and permissions

The backend defines 39 permissions (for example invoice.approve, payment.release_run, bank.approve_manual_match, audit.view). Table 3.2 summarises the six editable role templates. A unit test in the repository asserts that no template combines preparing and releasing payment runs and that the Admin template has no financial permissions.

Table 3.2: Role templates (from permissions.ts and README)

Role	Principal capabilities	Cannot
Requester	Create own invoices, view own, comment	Approve, pay
Accounts Team	Capture/code invoices, upload and review extraction, propose vendors and bank changes, prepare payment runs, hold payments, configure bank, import and reconcile, operational reports	Approve invoices, release payments, approve vendors

Role	Principal capabilities	Cannot
Approver	View assigned invoices; approve, reject, hold within limit; own reports	Create invoices, pay
Finance Head	All-invoice view, approve/reject/hold, approve vendors and bank changes, reveal bank number, release runs, release holds, confirm paid, approve manual matches, rules, tax settings, all reports, audit view	Prepare payment runs
Admin	Manage users, roles, organisation settings	Any financial action
Auditor	Read-only: invoices, vendors, payments, bank, all reports, audit view and export	Change anything

3.2 SOFTWARE AND HARDWARE REQUIREMENTS

The repository specifies the technology stack in its manifests and Compose file. It does not specify minimum hardware; the hardware figures in Table 3.4 are therefore **recommendations** for development, not repository requirements.

Table 3.3: Software stack (from package.json, docker-compose.yml and CI configuration)

Layer	Technology (as declared in the repository)
Client	React ^18.3.1, react-router-dom ^6.26, Recharts ^2.12, lucide-react, TypeScript ^5.5, Vite ^5.4
Server	Node.js (CI uses Node 20), Express ^4.21, TypeScript ^5.5, tsx (development), zod ^3.23
Database	MongoDB 6 (image mongo:6-jammy) as a single-node replica set rs0; Mongoose ^9.10
Queue/cache	Redis 7 (redis:7-alpine); BullMQ ^5.12; ioredis ^5.4; rate-limit-redis
Security libraries	bcryptjs, jsonwebtoken, helmet, cors, express-rate-limit, cookie-parser, google-auth-library
File and data handling	multer, pdfjs-dist ^3.11, exceljs, csv-parse, nodemailer; tesseract.js (optional dependency)
Packaging	Docker and Docker Compose; example production Compose, Caddyfile and nginx.conf
Testing / CI	node:test via tsx (unit and integration); GitHub Actions workflow with a real MongoDB replica set

Table 3.4: Recommended development environment (recommendation, not specified by the repository)

Item	Recommendation
Processor / memory	Dual-core or better CPU; 8 GB RAM (MongoDB, Redis, Node and Vite run together)
Storage	At least 10 GB free for Docker images and volumes
Operating system	Windows 10/11, macOS or Linux with Docker Desktop / Docker Engine
Runtime tools / network	Docker with Compose; Node.js 20 and npm for running without Docker; a modern browser; internet access to pull images (SMTP server and Google OAuth client only for those optional features)

CHAPTER 4
METHODOLOGY

This chapter presents the proposed architecture, its modules, the relationships between its entities and the sequence of operations.

4.1 SYSTEM ARCHITECTURE

InvoiceFlow follows a three-tier architecture (Figure 4.1). The React client communicates with the Express API over JSON. Each request passes through helmet, CORS, rate limiting, authentication (the bearer token is verified, then the user and the **current** role permissions are loaded from MongoDB), permission middleware and zod validation. Route handlers delegate to services, which use pure domain functions for decisions and Mongoose for persistence. Long-running or deferrable work (document extraction, statement matching, approval escalation, e-mail dispatch) is registered as BullMQ jobs; when queues are set to inline mode, as in tests, the same handlers run synchronously.

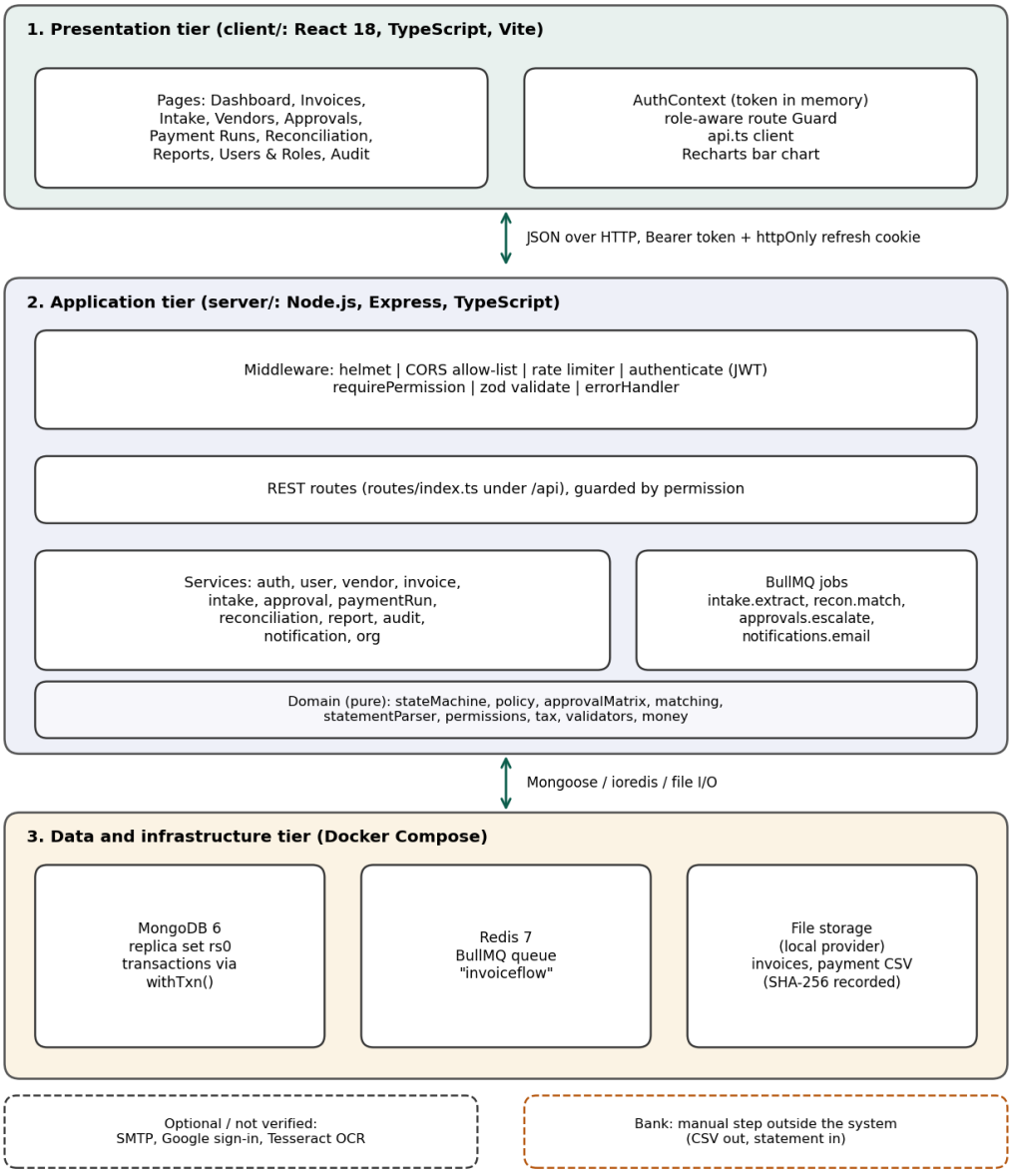


Figure 4.1: Three-tier architecture of InvoiceFlow, derived from the source layout and docker-compose.yml

Three architectural rules are visible throughout the code. First, **policy lives in the domain layer**: segregation-of-duties checks, approval-matrix selection and the matching engine are pure functions with no database access. Second, **mutations are transactional**: the helper withTxn runs a function inside a MongoDB transaction, and audit entries are written with the same session. Third, **tenant isolation is structural**: every model except the organisation itself is created with an organisation identifier, and a query that omits it throws.

4.2 MODULE DESCRIPTION

Table 4.1: Principal modules and their implementation files

Module	Responsibility	Main files
Authentication	Login with lockout, token issue, refresh rotation with reuse detection, invitation acceptance, Google sign-in for existing users	auth.service.ts, middleware/auth.ts
Users and roles	Invitations, activation, roles as permission sets, delegations, organisation settings	user.service.ts, org.service.ts, permissions.ts
Vendor management	Proposal/approval lifecycle, bank-change requests, encrypted bank number, masked display, audited reveal	vendor.service.ts, crypto.ts, validators.ts
Invoice intake	File-signature validation, PDF text extraction, field parsing and validation, vendor match by GSTIN, conversion to draft	intake.service.ts, domain/extraction/*, storage/*
Invoice management	Draft creation and editing, tax and TDS checks, duplicate control, comments, file attachment	invoice.service.ts, money.ts, tax.ts
Approvals	Rule matching, approver choice, step sequencing, decisions, hold/resume, delegation, escalation	approval.service.ts, domain/approvalMatrix.ts, policy.ts, stateMachine.ts
Payment runs	Run preparation, holds, cancellation, dual-signature release, CSV generation, manual confirmation, failure handling	paymentRun.service.ts, lib/csv.ts
Reconciliation	Bank accounts, mapping profiles, statement parsing and import, matching, manual match approval, classification	reconciliation.service.ts, domain/statementParser.ts, matching.ts
Reporting	Dashboard metrics, aging, vendor spend, turnaround, payments due, cash flow, TDS, reconciliation status, journal export	report.service.ts, client Dashboard.tsx, Reports.tsx
Audit logging	Hash-chained append-only entries; chain verification; export	audit.service.ts, models/system.ts

Invoice lifecycle. An invoice has nine statuses (draft, in_review, approved, rejected, on_hold, scheduled, released, paid, reconciled). The only way to change status is a named action in the state machine (Figure 4.2); there is no generic status-update endpoint.

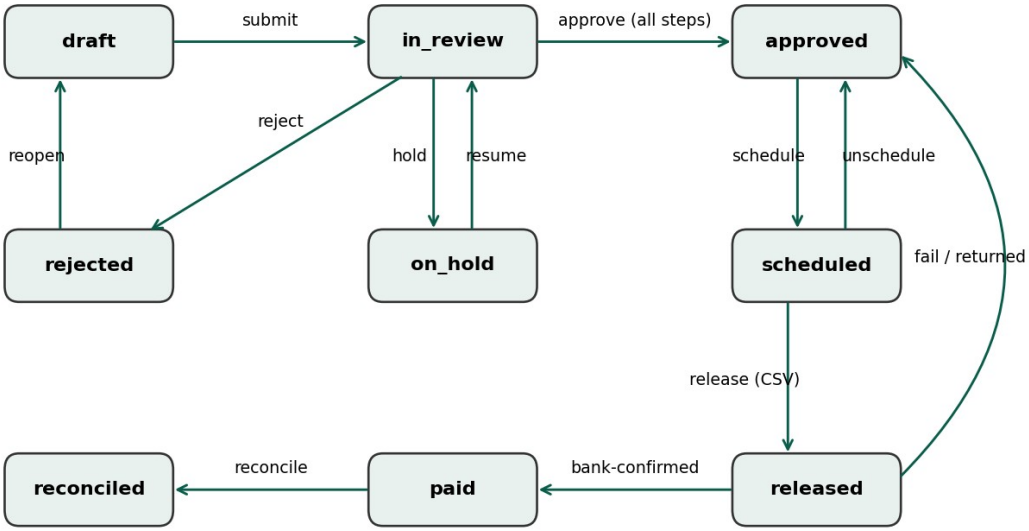


Figure 4.2: Invoice status transitions defined in domain/stateMachine.ts

4.3 CLASS DIAGRAM

Figures 4.3 to 4.5 present the entity-relationship (class) view of the Mongoose models in models/core.ts, finance.ts, payments.ts, bank.ts and system.ts. Only principal fields are shown; the Counter, RefreshToken, Invitation, Notification, InvoiceComment and AuditLog helper collections are omitted except where noted. Relationships are references by ObjectId (not embedded documents), except the vendor bank block, the line items and the file reference, which are embedded.

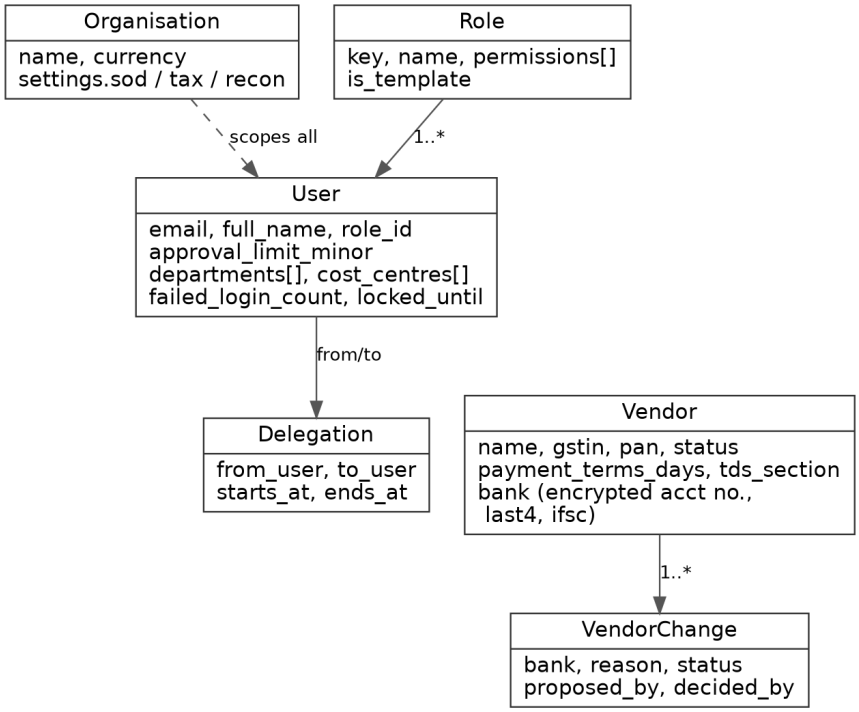


Figure 4.3: Entity relationships: organisation, users, roles, delegation and vendors

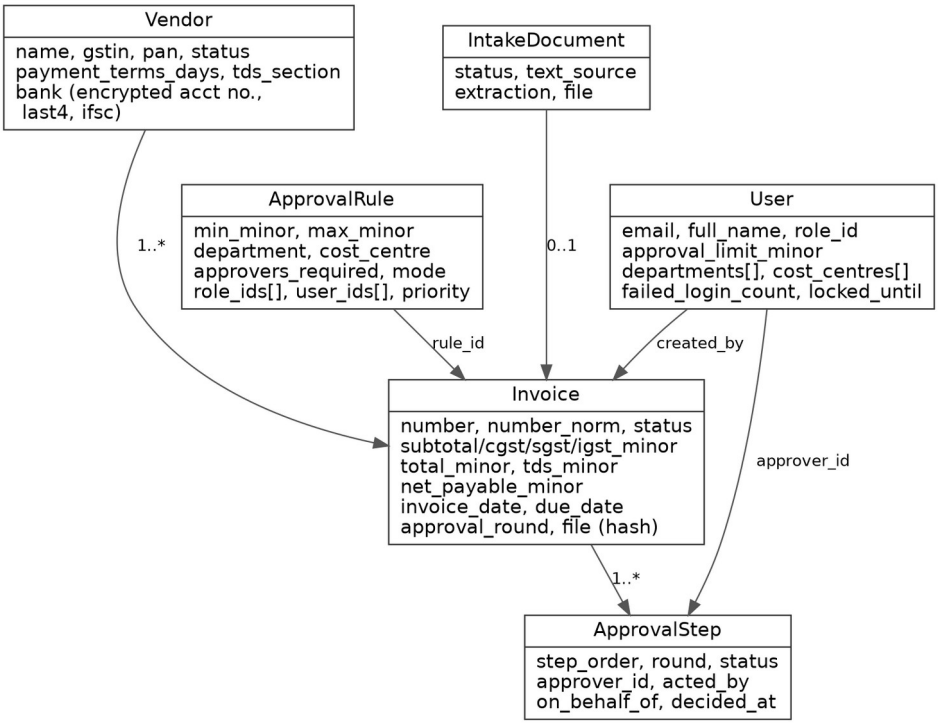


Figure 4.4: Entity relationships: invoices, intake documents, approval rules and approval steps

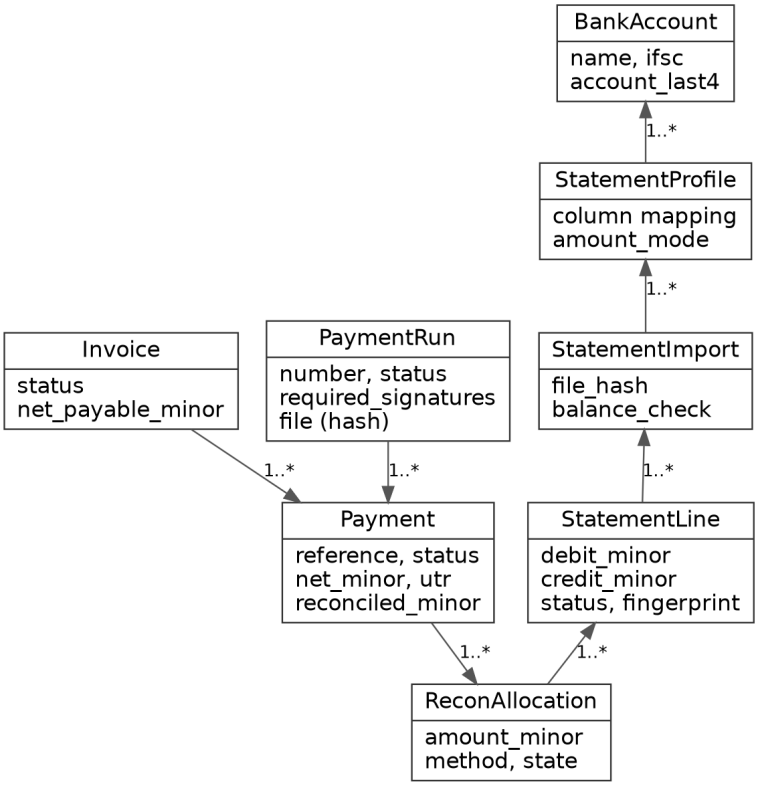


Figure 4.5: Entity relationships: payment runs, payments and bank reconciliation (AuditLog is append-only and references entities by type and id)

4.4 SEQUENCE DIAGRAM

Figure 4.6 shows a representative end-to-end workflow: invoice creation and submission, approval, payment-run preparation, release and CSV generation, the manual upload of that CSV to a bank (outside the system), and the later statement import and reconciliation. The dashed arrow marks the step that InvoiceFlow does **not** perform. The optional confirm-paid call is the manual alternative to statement-based confirmation.

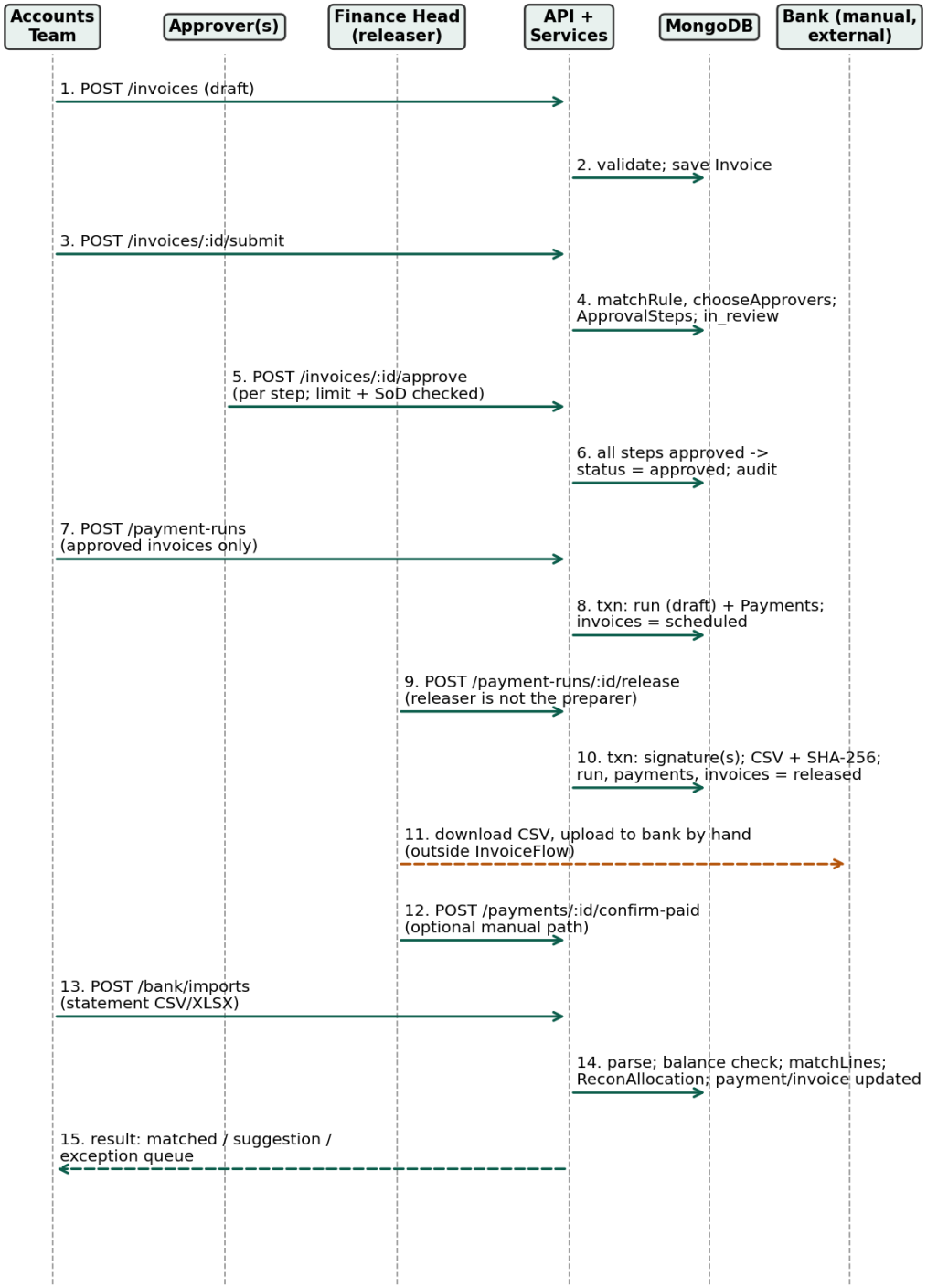


Figure 4.6: Sequence diagram of the invoice-to-reconciliation workflow

CHAPTER 5
RESULTS AND DISCUSSION

This chapter presents the implementation, the key functions of the system and the status of its testing and verification.

5.1 IMPLEMENTATION OVERVIEW

The client and server are both written in TypeScript. On the server, the financial logic is carried by five Mongoose model files, twelve services and seven domain modules (plus an extraction sub-package). Verification evidence for this report comes from three different sources and is deliberately kept separate:

- **Static code inspection** – the source was read to describe behaviour (all of Chapter 4 and Section 5.2).
- **Automated tests present in the repository** – 31 unit tests and 43 integration tests exist (counted from the test files). The README states they passed in the author's build environment, which used a MongoDB-compatible stand-in without transactions. They were **not re-executed** while preparing this report because project dependencies were not installed and there was no network access.
- **Runtime observation** – the author's own 15 screenshots in docs/screenshots (taken from a running instance with the earlier interface) show a released payment run, a statement import with an automatic match, and an audit-chain verification message. They are not reproduced in this report. The figures in this report are sample-data renderings of the final interface and are **not** runtime evidence.

Dashboard and reports. The dashboard (Dashboard.tsx, backed by reports.dashboard) shows invoice counts and totals by status, overdue amount, approvals waiting for the current user, payment and reconciliation counters, and average approval time. In the delivered version the client has a small shared chart module (components/charts.tsx) built on Recharts, with three components: vertical bars, horizontal ranked bars and a proportional status bar. The Dashboard shows an invoices-by-status bar and the weekly cash-need chart; the Reports page adds charts for AP aging, cash-flow, vendor spend, approval turnaround, TDS by section and

reconciliation outcomes, each above its data table. Charts follow a restrained black-and-red style and carry a text description for screen readers. The Reports page also supports CSV download for several reports, with formula-injection neutralisation covered by a unit test. **The charts never contain data of their own**: they draw whatever the report endpoints return from MongoDB. The chart figures in this report (A.13 to A.15) were rendered from the real client code in a headless browser using sample data supplied by a test harness, because no database could be run while preparing the report; they illustrate the design only and are not captured from a live system.

Validation. Requests are validated with zod schemas (routes/schemas.ts). Domain validators check GSTIN structure, state code and check character, PAN and IFSC patterns, and normalise invoice numbers for duplicate detection. Per the README, the GSTIN check-digit routine was validated against a single published sample and a failing check digit raises a warning rather than a rejection.

5.2 KEY FUNCTIONS (CODE EXPLANATION)

The excerpts below are shortened copies of real source code (omissions are marked with comments). Each is followed by its purpose, inputs, processing and outputs.

5.2.1 Invoice state machine (domain/stateMachine.ts)

```
export const INVOICE_ACTIONS = {
  submit:    { from: ['draft'], to: 'in_review' },
  approve:   { from: ['in_review'], to: 'approved' },
  reject:    { from: ['in_review'], to: 'rejected' },
  hold:      { from: ['in_review'], to: 'on_hold' },
  schedule:  { from: ['approved'], to: 'scheduled' },
  release:   { from: ['scheduled'], to: 'released' },
  paid:      { from: ['released'], to: 'paid' },
  reconcile: { from: ['paid'], to: 'reconciled' },
  // ... resume, reopen, unschedule, fail omitted here
} as const;

export function assertTransition(from: InvoiceStatus, action:
InvoiceAction): InvoiceStatus {
  const to = nextStatus(from, action);
  if (!to) throw new Error(`Cannot ${action} an invoice that is $
{from}.`);
  return to;
}
```

Listing 5.1: Legal invoice actions and transition guard

Purpose: guarantee that an invoice can only move along defined edges. **Input:** the current status and a named action. **Processing:** nextStatus looks up the action and returns the target only if the current status is in its from-list; assertTransition throws otherwise. **Output:** the new status or an error. Services call assertTransition before saving, which is why an unpaid invoice cannot be marked reconciled.

5.2.2 Segregation-of-duties policy (domain/policy.ts)

```
export function checkRunRelease(sod: SodSettings, actorId: string,
run: { prepared_by: any; signatures?: { user_id: any }[] }):
Violation | null {
  if (sod.run_preparer_cannot_release && String(run.prepared_by) ===
String(actorId))
    return { rule: 'run_preparer_cannot_release', message: 'You
prepared this payment run, so you cannot release it.' };
  if ((run.signatures || []).some((s) => String(s.user_id) ===
String(actorId)))
    return { rule: 'distinct_signatories', message: 'You have
already signed this release. ...' };
  return null;
}

export function requiredSignatures(sod: SodSettings, runNetMinor:
number): number {
  return sod.dual_release_threshold_minor !== null && runNetMinor >=
sod.dual_release_threshold_minor ? 2 : 1;
}
```

Listing 5.2: Release checks and signature threshold

Purpose: enforce maker-checker release. **Input:** SoD settings, the acting user and the run. **Processing:** the first check rejects the run's preparer, the second rejects a user who has already signed; requiredSignatures returns 2 when the run's net amount reaches the configured threshold (default 25,00,000 rupees, expressed in paise) and 1 otherwise. **Output:** null when allowed, otherwise a rule name and message returned to the client as a 403 SOD_VIOLATION.

5.2.3 Choosing approvers (domain/approvalMatrix.ts)

```
const eligible = candidates.filter((c) =>
  c.active && c.can_approve && !ex.has(String(c.id)) &&
  (rule.user_ids.length === 0 || rule.user_ids.includes(c.id)) &&
  (rule.role_ids.length === 0 || rule.role_ids.includes(c.role_id))
&&
  inScope(c.departments, ctx.department) && inScope(c.cost_centres,
ctx.cost_centre) &&
  (c.limit_minor === null || c.limit_minor >= ctx.total_minor));
if (eligible.length < rule.approvers_required) {
  return { ok: false, reason: `Rule "${rule.name}" needs $
{rule.approvers_required} approver(s) ...` };
}
eligible.sort((a, b) => (a.limit_minor ?? Infinity) - (b.limit_minor
?? Infinity) || a.id.localeCompare(b.id));
```

Listing 5.3: Eligibility filter for approvers

Purpose: pick approvers who are truly entitled to approve the entire invoice. **Input:** the matched rule, candidate users and the invoice context. **Processing:** candidates must be active, hold invoice.approve, not be excluded (creator or editors), match the rule's users and roles, be in department and cost-centre scope, and have a limit at least equal to the invoice total. If too few remain, the invoice cannot be submitted. Sequential steps are ordered by ascending limit. **Output:** ordered approval steps or a failure reason.

5.2.4 Payment-run release and CSV generation (services/paymentRun.service.ts)

```
export const genericCsvExporter: PaymentFileExporter = {
  id: 'generic_csv',
  generate(rows, runNumber) {
    const content = toCsv(['reference', 'beneficiary_name',
'account_number', 'ifsc', 'amount', 'currency', 'narration',
'pay_date'],
      rows.map((r) => [r.reference, r.beneficiary_name,
r.account_number, r.ifsc, major(r.amount_minor), r.currency,
r.narration, r.pay_date.toISOString().slice(0, 10)]));
    return { filename: `${runNumber}.csv`, content, mime: 'text/csv'
};
  },
};
// inside releaseRun(), within withTxn():
run.signatures.push({ user_id: actor.userId, at: new Date() });
if (run.signatures.length < run.required_signatures) { /* save,
audit, return 'awaiting_signature' */ }
const out = exporter.generate(rows, run.number);
const saved = await storage().save(Buffer.from(out.content, 'utf8'),
out.filename, out.mime, `${actor.orgId}/payment-files`);
for (const p of live) { p.status = 'released'; p.released_at = now;
await p.save({ session }); }
```

Listing 5.4: Generic CSV exporter and release flow

Purpose: prepare the bank-upload file after the required signatures. **Input:** a draft run and the releasing user. **Processing:** inside one transaction the signature is recorded; if more signatures are required the function returns "awaiting_signature". Otherwise held payments are cancelled and un-scheduled, the beneficiary account number is decrypted only to build the rows, the CSV is generated and stored with its SHA-256 hash, and live payments and invoices become "released". **Output:** the run, state "released", and a downloadable file. **Important:** no bank API is called; the payout provider is a stub that throws NOT_CONFIGURED, and payments become "paid" only through confirmPaid or statement matching.

5.2.5 Reconciliation matching (domain/matching.ts)

```
// Rule 1: our payment reference (or bank UTR) appears on the bank
line.
const refCands = open.filter((p) => (norm(p.reference) &&
nn.includes(norm(p.reference))) ||
  (p.utr && (refN === norm(p.utr) || nn.includes(norm(p.utr)))));
if (refCands.length === 1) {
  const p = refCands[0]; const o = outstanding(p);
  if (Math.abs(o - debit) <= tol) {
    const al = [{ line_id: line.id, payment_id: p.id, amount_minor:
Math.min(debit, o) }]; reserve(al);
    out.push({ kind: 'match', allocations: al, method: 'reference',
confidence: 0.99 });
  } else if (debit < o) { /* partial: method 'reference_partial',
confidence 0.9 */ }
  else { /* exception: 'amount_exceeds_payment' */ }
  continue;
}
```

Listing 5.5: Matching rule 1 (payment reference or UTR on the bank line)

Purpose: automatically link a bank debit to a released payment when the evidence is strong. **Input:** parsed statement lines, open payments and tolerance settings. **Processing:** rules are tried in order: (1) reference/UTR, (2) exact amount plus vendor-name tokens, (3) exact amount within a date window (only a suggestion), (4) one debit covering a subset of a run (suggestion). Ambiguity becomes an exception; credits are never matched to vendor payments. **Output:** proposals of kind match, suggestion, exception or classified, which the service persists as ReconAllocation records. Manual matches require a second person.

5.2.6 Hash-chained audit log (services/audit.service.ts)

```
function digest(prev: string, e: { seq: number; org: string; actor:
string | null; action: string; /* ... */ at: Date }) {
  return sha256(prev + stableStringify({ ...e, at:
e.at.toISOString() }));
}
// writeAudit(): seq from an atomic Counter; prev = hash of entry
seq-1 (or 'GENESIS')
const hash = digest(prev, body);
await AuditLog.create([{ organisation_id: actor.orgId, seq,
prev_hash: prev, hash, /* ... */ }], { session });

// verifyAuditChain(): recompute every entry in order
if (d.prev_hash !== prev) return { ok: false, checked, broken_at:
d.seq, reason: 'prev_hash does not match the previous entry.' };
if (h !== d.hash) return { ok: false, checked, broken_at: d.seq,
reason: 'Entry contents do not match its hash (the record was
altered).' };
```

Listing 5.6: Audit entry hashing and chain verification

Purpose: make later alteration detectable. **Processing:** each entry receives a sequence number from an atomic counter and stores the hash of the previous entry; the hash covers the actor, action, entity, old and new values and timestamp. verifyAuditChain recomputes the chain in order. **Output:** ok or the first broken sequence number. The log model also uses an append-only plugin that refuses updates. This detects tampering after the fact; it does not prevent a database administrator from rewriting the entire chain.

5.2.7 Authentication and permission middleware (middleware/auth.ts)

```
export const authenticate = async (req, _res, next) => {
  const h = req.headers.authorization;
  if (!h || !h.startsWith('Bearer ')) throw
unauthorized('Authentication required.');
  const claims = verifyAccess(h.slice(7));
  const user = await User.findOne({ _id: claims.sub,
organisation_id: claims.org });
  if (!user || !user.is_active || user.token_version !== claims.tv)
throw unauthorized('Session is no longer valid.');
  const role = await Role.findOne({ _id: user.role_id,
organisation_id: claims.org }).lean();
  req.actor = { orgId: claims.org, userId: String(user._id), /* ...
*/ permissions: new Set(role?.permissions || []) };
};
export const requirePermission = (...perms: Permission[]) => (req,
_res, next) => {
  const missing = perms.filter((p) => !
req.actor!.permissions.has(p));
  if (missing.length) return next(forbidden(`Missing permission: $
{missing.join(', ')}`, 'MISSING_PERMISSION', { required: perms }));
  next();
};
```

Listing 5.7: Token verification and permission guard

Purpose: authorise on current permissions. **Processing:** the bearer token is verified, the user is loaded and checked for active status and token version, the role is loaded and its permissions become the actor's permission set. requirePermission requires all listed permissions; a companion requires any. **Output:** req.actor, or a 401/403 error. Because permissions are read per request, editing a role takes effect immediately.

5.2.8 Transactions and money handling (lib/txn.ts, lib/money.ts)

```
export async function withTxn<T>(fn: (session: ClientSession |
undefined) => Promise<T>): Promise<T> {
  if (env.TXN_DISABLED) return fn(undefined);   // tests only;
enforced in env.ts
  const session = await mongoose.startSession();
  try {
    let result: T;
    await session.withTransaction(async () => { result = await
fn(session); });
    return result!;
  } finally { await session.endSession(); }
}
```

Listing 5.8: Transaction helper

```
// lib/money.ts : all money is an integer number of paise
export function parseMajorToMinor(input: string | number): number {
  // handles "1,23,456.78", "₹ 4,54,300", "(250.00)"; throws on
garbage
  if (!/^\d+(\.\d+)?$/.test(s)) throw new Error(`Invalid amount: $
{input}`);
  const minor = parseInt(whole, 10) * 100 + parseInt((frac +
'00').slice(0, 2), 10);
  if (!Number.isSafeInteger(minor)) throw new Error('Amount too
large');
  return neg ? -minor : minor;
}
```

Listing 5.9: Parsing human amounts into integer paise

withTxn gives service functions an all-or-nothing boundary; it requires a replica set, and the TXN_DISABLED switch exists only for tests. parseMajorToMinor accepts Indian digit grouping and parentheses for negatives, rejects more than two significant decimals, and returns a safe integer, so floating-point error cannot enter stored amounts.

5.3 TESTING AND VERIFICATION STATUS

Table 5.1 lists representative test cases. The "Actual result" column records only what could be established for this report. The abbreviation **REPORTED** means the author's README claims the test passed in the author's environment and this report has not re-run it; **OBSERVED** means a screenshot shows the behaviour in a running application; **NOT VERIFIED** means no execution evidence is available.

Table 5.1: Testing and verification status

No.	Test case	Expected behaviour	Actual result / evidence	Status
1	Money parsing and formatting (money_validators.test.ts)	Amounts become integer paise; no float error	Test exists; not re-executed here. README: unit suite passed	REPORTED
2	GSTIN/PAN/IFSC validators	Valid formats accepted, invalid rejected	Test exists; not re-executed. README states GSTIN check digit validated on one sample only	REPORTED
3	Policy, state machine and approval matrix (policy_state_matrix.test.ts)	Illegal transitions and SoD violations refused; approvers cover full amount	Test exists; not re-executed	REPORTED
4	Matching engine (matching.test.ts)	Reference match auto-applies; ambiguity to exceptions; credits ignored	Test exists; not re-executed	REPORTED
5	Statement parser (statement_extraction.test.ts)	Strict dates, separate debit/credit, balance check, duplicate fingerprints	Test exists; not re-executed	REPORTED
6	Login lockout, token rotation, permission checks (auth_roles.test.ts)	Lockout after repeated failure; reused refresh token revokes session	Test exists; run on a database stand-in per README; not re-executed	REPORTED

No.	Test case	Expected behaviour	Actual result / evidence	Status
7	Approval flow, delegation, visibility (finance_flow.test.ts)	Two-step approval; no self-approval; delegate recorded	Test exists; not re-executed	REPORTED
8	Payment run, dual release, bank import (payments_bank.test.ts)	Preparer cannot release; held payments carved out; file hashed; imports matched	Test exists; not re-executed	REPORTED
9	Complete UI workflow: invoice, two approvals, run, release, import, match, audit verify	Invoice reaches reconciled; audit chain intact	Author's screenshots in docs/screenshots (earlier interface, running instance) show the workflow stages; one shows "Chain verified: 20 entries are intact". Environment used a stand-in database without transactions. Not re-run in the final interface	OBSERVED (author's screenshots)
10	Login page and role-restricted navigation	Sign-in form shown; menu limited by role	Seen in the author's screenshots; the final interface was rendered only with sample data (A.1 to A.16)	OBSERVED (partly)
11	Real multi-document transactions on a MongoDB replica set	Atomic rollback on failure	CI workflow is configured; no execution result available	NOT VERIFIED
12	Partial unique indexes (vendor GSTIN, payment idempotency key)	Duplicates rejected by the database	Not exercised on a real MongoDB (per README)	NOT VERIFIED
13	Docker Compose start-up of all services	Mongo, Redis, server, client healthy	Compose file inspected only; not executed here	NOT VERIFIED
14	Redis / BullMQ background processing	Jobs retried and processed by workers	Configured; tests run jobs inline	NOT VERIFIED
15	SMTP delivery, Google sign-in, OCR	Mail sent; Google user signed in; image text read	Configured as optional; no result available	NOT VERIFIED
16	Bank-specific file formats and payout	—	Not implemented; payout provider returns NOT_CONFIGURED	NOT IMPLEMENTED
17	Dashboard and report charts render (components/charts.tsx)	Bars and status mix drawn from report data	Rendered in headless Chromium with sample data from a test harness; not run against live report output (A.13 to A.15)	OBSERVED (sample data)
18	New vendor-management and report-shape tests (vendor_management, report_charts)	Edit, deactivate guards, stats; report fields match chart inputs	Tests written; no database available to run them	NOT VERIFIED

Discussion. The evidence supports the claim that the core workflow was exercised end to end in a development environment, and that the rule engines have automated tests. It does not support claims about production readiness, real transactional behaviour, queue processing or external integrations. The README itself recommends running the integration tests with TEST_REAL_TXN=1 against the Docker MongoDB before relying on the system; that step remains to be done.

CHAPTER 6
CONCLUSION & FUTURE WORK

This chapter presents the conclusions drawn from the project and the enhancements proposed for future work.

6.1 CONCLUSION

InvoiceFlow demonstrates that an accounts-payable workflow can be implemented as a controlled state machine with enforceable segregation of duties, explainable approvals, evidence-based payment confirmation and a tamper-evident audit trail. The implemented system covers invoice capture and validation, vendor and bank-detail control, rule-driven approvals, payment-run preparation with CSV generation, statement import and reconciliation, reporting, and permission-based access control, on a React/TypeScript, Express, MongoDB, Redis and Docker stack.

The principal limitations are as follows. No real payment is made: the system produces a generic CSV and the bank step is manual. Real MongoDB transactions, Redis queues, SMTP, Google sign-in and OCR have not been verified in this report; the automated tests were not re-run for it; the screenshots come from an environment without transactions. There is no bank-specific file format, no built-in tax rates, no MFA, no purchase-order matching. Tax settings and statement layouts must be configured and validated by the user, ideally with an accountant, before any real use.

6.2 FUTURE ENHANCEMENTS

The following items are **future work** and are not implemented:

- Run the integration suite against a real replica set (TEST_REAL_TXN=1) and add continuous execution in CI; record the results.
- Bank-specific payment-file formats and payout-provider integration based on official bank specifications, possibly aligned with standards such as ISO 20022 [20].
- Multi-factor authentication (the user model already contains placeholder fields).
- Accounting-software integration beyond the generic journal CSV; purchase-order and goods-receipt matching.
- Enhanced reconciliation: bank-specific statement layouts validated on real statements, partial-payment workflows, and learning classifiers.

- Further visualisations such as trends over time, end-to-end browser tests, and an accuracy study of invoice extraction using the provided accuracy script.
- Production deployment hardening: managed MongoDB and Redis, HTTPS using the example Caddy configuration, backups of the database and of the encryption key, monitoring, and an independent security review.

APPENDIX

SOURCE CODE

1. Project Structure

```
proj/
├── docker-compose.yml, docker-compose.prod.example.yml
├── Caddyfile.example, .env.example
├── .github/workflows/ci.yml
├── docs/screenshots/        (15 PNG screenshots)
├── samples/                 (placeholders only)
├── client/
│   ├── Dockerfile, Dockerfile.prod, nginx.conf, vite.config.ts
│   └── src/
│       ├── App.tsx, main.tsx
│       ├── context/AuthContext.tsx
│       ├── components/   (Layout, InvoiceForm, ui, charts)
│       ├── lib/          (api, hooks, format)
│       ├── styles/app.css
│       └── pages/
│           Landing, Login, Dashboard, Invoices, Intake,
│           Vendors, Approvals, ApprovalRules, PaymentRuns,
│           Reconciliation, Reports, UsersRoles, AuditLog,
│           Settings, AcceptInvite
└── server/
    ├── Dockerfile, Dockerfile.prod, package.json
    └── src/
        ├── index.ts, app.ts
        ├── config/       (env, database, redis)
        ├── middleware/   (auth, validate, rateLimiter,
        │                  errorHandler)
        ├── routes/       (index.ts, schemas.ts)
        ├── services/     (auth, user, vendor, invoice, intake,
        │                  approval, paymentRun, reconciliation,
        │                  report, audit, notification, org)
        ├── domain/       (stateMachine, policy, approvalMatrix,
        │                  permissions, matching,
        │                  statementParser, tax, extraction/)
        ├── models/       (core, finance, payments, bank, system)
        ├── queues/, workers/, lib/ (money, crypto, csv, txn)
        ├── scripts/      (bootstrap-admin, seed-dev,
        │                  seed-demo-flow, extraction-accuracy)
        └── tests/        (unit/, integration/, helpers/)
```

2. Dependencies and Configuration

Dependencies are listed in Table 3.3. Environment variables recognised in .env.example and docker-compose.yml are listed below (values are secrets or deployment-specific and are not reproduced).

Table A.1: Configuration variables

Variable	Purpose
JWT_SECRET	Required; signs access tokens (at least 32 characters; weak placeholders rejected)
ENCRYPTION_KEY	Required; 32 random bytes in base64 for AES-256-GCM encryption of vendor bank accounts; loss makes stored numbers unreadable
MONGO_URI, REDIS_HOST, REDIS_PORT	Database and queue connection (set by Compose; replica set rs0)
CORS_ORIGIN, APP_BASE_URL	Allowed browser origin and link base
UPLOAD_DIR, MAX_FILE_SIZE	Local file storage location and upload limit (10,485,760 bytes in Compose)
GOOGLE_CLIENT_ID	Optional; Google sign-in for already-invited users
SMTP_HOST/PORT/ USER/PASS/FROM	Optional; without them notifications stay in-app, marked undelivered
OCR_ENABLED	Optional; OCR for PNG/JPEG only
SEED_PASSWORD	Development seed only; refuses to run in production

3. MongoDB Models and Schema Example

Models are defined with Mongoose and an organisation-scoping helper (models/plugins.ts). Collections: Organisation, Role, User, Invitation, RefreshToken, Delegation, Counter, Vendor, VendorChange, Invoice, InvoiceComment, ApprovalRule, ApprovalStep, PaymentRun, Payment, BankAccount, StatementProfile, StatementImport, StatementLine, ReconAllocation, AuditLog, Notification, IntakeDocument. The Payment schema (shortened) is shown in Listing A.1.

```
export const Payment = model('Payment', (() => {
  const s = new Schema({
    run_id: oid('PaymentRun', true), invoice_id: oid('Invoice',
true), vendor_id: oid('Vendor', true),
    reference: { type: String, required: true }, vendor_name:
String, invoice_number: String,
    gross_minor: minor(true), tds_minor: minor(true), net_minor:
minor(true),
    status: { type: String, enum:
['scheduled','held','released','paid','failed','returned','cancelled
'], default: 'scheduled', index: true },
    paid_via: { type: String, enum: ['bank_statement', 'manual'] },
    bank_reference: { type: String, default: null }, utr: { type:
String, default: null },
    reconciled_minor: { ...minor(), default: 0 }, reconciled:
{ type: Boolean, default: false },
    // ... hold, failure and beneficiary fields omitted
  }, { timestamps: true });
  s.index({ organisation_id: 1, reference: 1 }, { unique: true });
  s.index({ organisation_id: 1, idempotency_key: 1 }, { unique:
true, partialFilterExpression: { idempotency_key: { $type:
'string' } } });
  return s;
})());
```

Listing A.1: Payment schema (models/payments.ts, shortened)

4. API Request and Response Examples

All routes are mounted under /api and require a bearer token unless stated. The examples below follow the zod schemas and DTO functions in the source; **identifier and amount values are illustrative placeholders, not captured output**.

```
POST /api/payment-runs          (permission: payment.prepare_run)
{ "invoice_ids": ["<invoiceObjectId>"], "pay_date": "2026-10-30",
"notes": "Weekly run" }

201 Created  -> run DTO (fields include)
{ "id": "...", "number": "PR-2026-0001", "status": "draft", "count":
1,
  "gross_minor": 0, "tds_minor": 0, "net_minor": 0,
  "required_signatures": 1, "signatures": [], "has_file": false }
```

Listing A.2: Prepare a payment run

```
POST /api/payment-runs/:id/release   (permission:
payment.release_run)

200 OK -> { "run": { ... "status": "released" }, "state":
"released", "idempotent": false }
         or { "state": "awaiting_signature", "signatures_needed": 1,
... }

403 Forbidden (preparer attempts release)
{ "error": "You prepared this payment run, so you cannot release
it.",
  "code": "SOD_VIOLATION", "details": { "rule":
"run_preparer_cannot_release" },
  "request_id": "..." }
```

Listing A.3: Release a run and a segregation-of-duties error

```
POST /api/payments/:id/confirm-paid   (permission:
payment.confirm_paid)
{ "bank_reference": "<bank reference>", "paid_date": "2026-11-02" }
200 OK -> payment DTO with "status": "paid"   (only from "released")
```

Listing A.4: Manual payment confirmation

5. Selected Source-Code Excerpts

Representative excerpts appear in Section 5.2 (Listings 5.1–5.9) and Appendix item 3 (Listing A.1).

6. Security and Permission Mechanisms

Table A.2: Security mechanisms present in the source

Mechanism	Implementation
Password storage and lockout	bcrypt hashing; failed_login_count and locked_until on User; generic login error
Sessions	Short-lived JWT access token; rotating refresh token (hash stored) in httpOnly cookie; token_version invalidates sessions; header requirement on refresh/logout
Authorisation	requirePermission / requireAnyPermission; permissions loaded per request; Guard component in client is convenience only
Segregation of duties	policy.ts functions: no self-approval, run preparer cannot release, proposer cannot approve, importer cannot approve manual matches, distinct signatories
Data protection	AES-256-GCM for bank account numbers; masked display (last four digits); reveal requires vendor.bank_reveal and is audited
Input and upload safety	zod validation; JSON body limit 1 MB; uploads validated by file signature; authenticated file endpoints; CSV formula-injection neutralisation; payment file hash re-checked on download
Transport and abuse controls	helmet; CORS origin allow-list; express-rate-limit (Redis store when available); HTTPS expected via reverse proxy (example only)
Audit	Hash-chained append-only log; /api/audit/verify

7. Important Application Routes

Table A.3: Selected API routes (prefix /api) and required permissions

Method	Path	Permission
POST	/auth/login, /auth/refresh, /auth/logout	none (rate-limited); refresh needs cookie
POST	/users/invite	user.manage
GET/ POST	/roles, /roles (PATCH/DELETE :id)	role.manage (list also user.manage, rules.manage)
POST	/vendors; /vendors/:id/approve	vendor.propose; vendor.approve
POST	/vendors/:id/bank-changes; /vendors/bank-changes/:id/approve	vendor.bank_change.propose; vendor.bank_change.approve
POST	/vendors/:id/reveal-bank	vendor.bank_reveal
POST	/intake/upload; /intake/:id/create-invoice	invoice.upload; invoice.review_extraction
GET/ POST	/invoices; /invoices/:id/submit	view or create permissions; submit via invoice view
POST	/invoices/:id/approve | reject | hold	invoice.approve | invoice.reject | invoice.hold
GET	/approvals/queue	invoice.approve / reject / hold
POST	/approval-rules (PUT/DELETE :id)	rules.manage
POST	/payment-runs; /payment-runs/:id/release	payment.prepare_run; payment.release_run
GET	/payment-runs/:id/file	payment.release_run
POST	/payments/:id/hold | release-hold | confirm-paid | mark-failed	payment.hold | payment.release_hold | payment.confirm_paid | payment.confirm_paid
POST	/payment-runs/:id/send-to-bank	payment.release_run – always returns NOT_CONFIGURED
POST	/bank/imports; /bank/run-matching	bank.import; bank.reconcile
POST	/bank/lines/:id/match | approve-match | unmatch	bank.reconcile | bank.approve_manual_match | bank.approve_manual_match
GET	/reports/dashboard; /reports/aging; /reports/tds	authenticated; report.view_ops/all; report.view_all
GET	/audit; /audit/verify	audit.view

SCREENSHOTS OF THE PROJECT

All figures below show the final Swiss Modernism interface. They were rendered from the real client code in a headless browser using **sample data** (organisation "Demo Foods Pvt Ltd") supplied by a test harness, because no database could be run while preparing this report. They show the screen design and layout, not output captured from a live system. The author's earlier screenshots of a running instance (earlier interface) are in the project folder docs/screenshots.

Desktop views

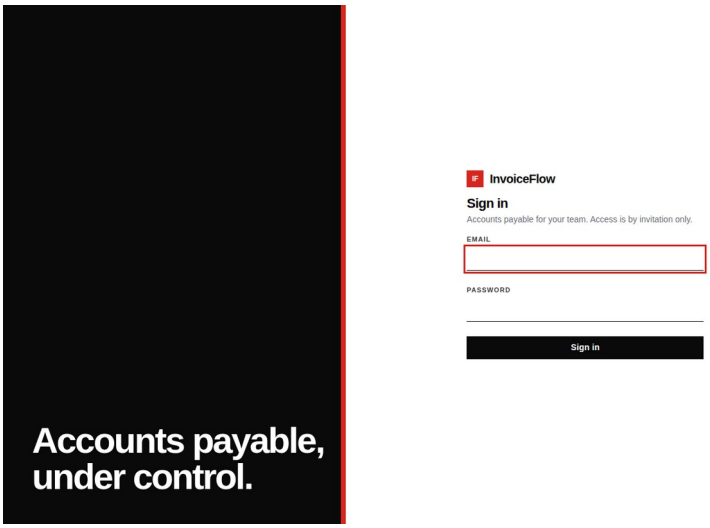


Figure A.1: Sign-in page (entry point of the application)

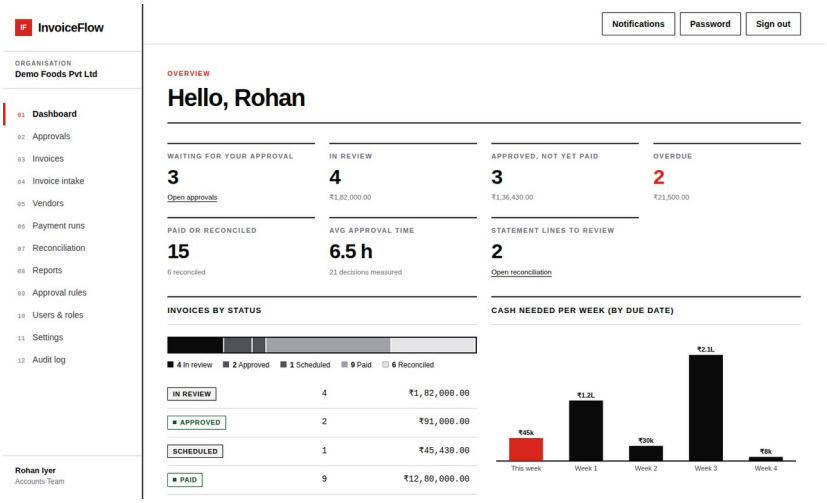


Figure A.2: Dashboard: status mix, key figures and weekly cash-need chart

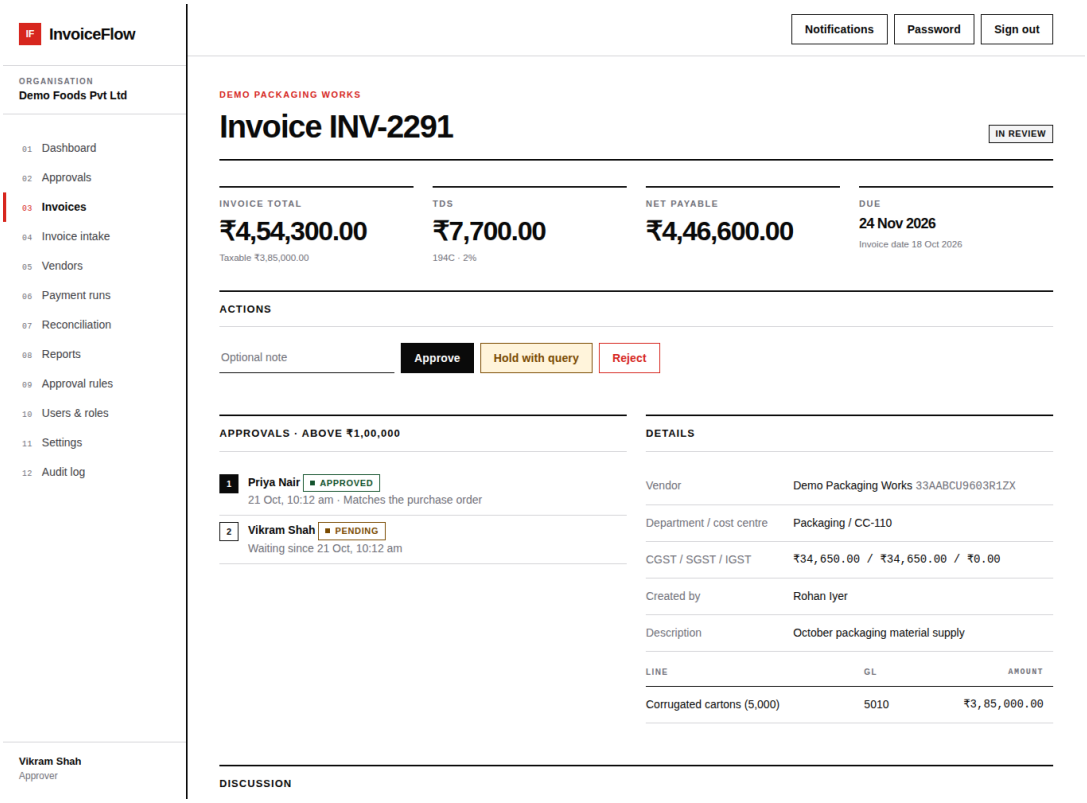


Figure A.3: Invoice INV-2291 in review with two sequential approval steps

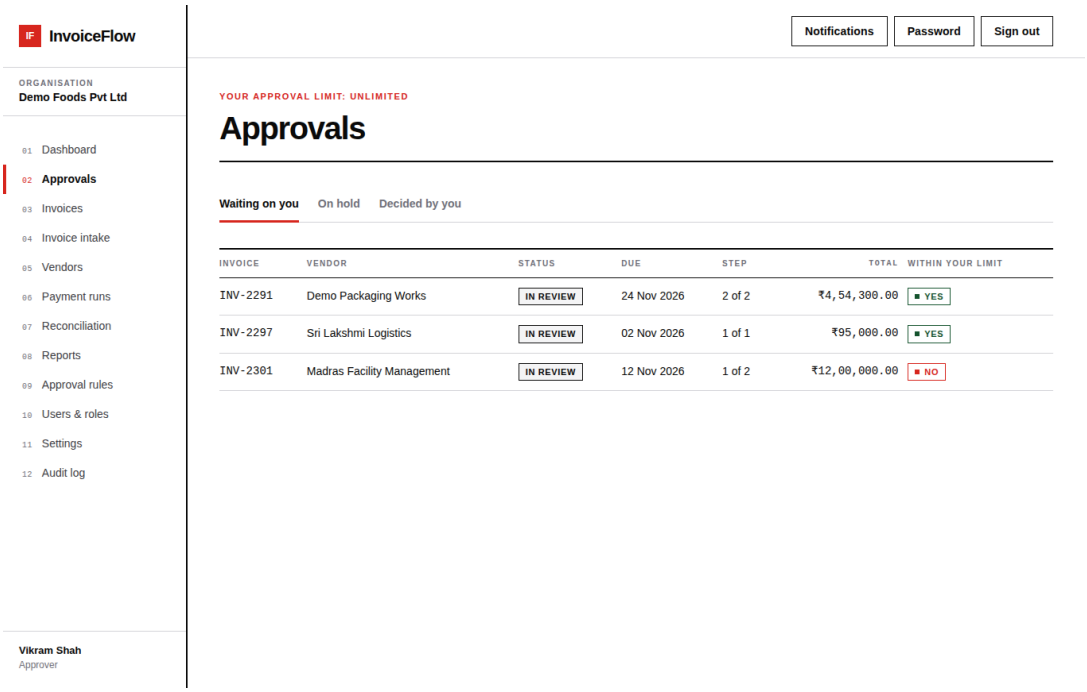


Figure A.4: Approver's queue showing invoices waiting on the user and the approval limit check

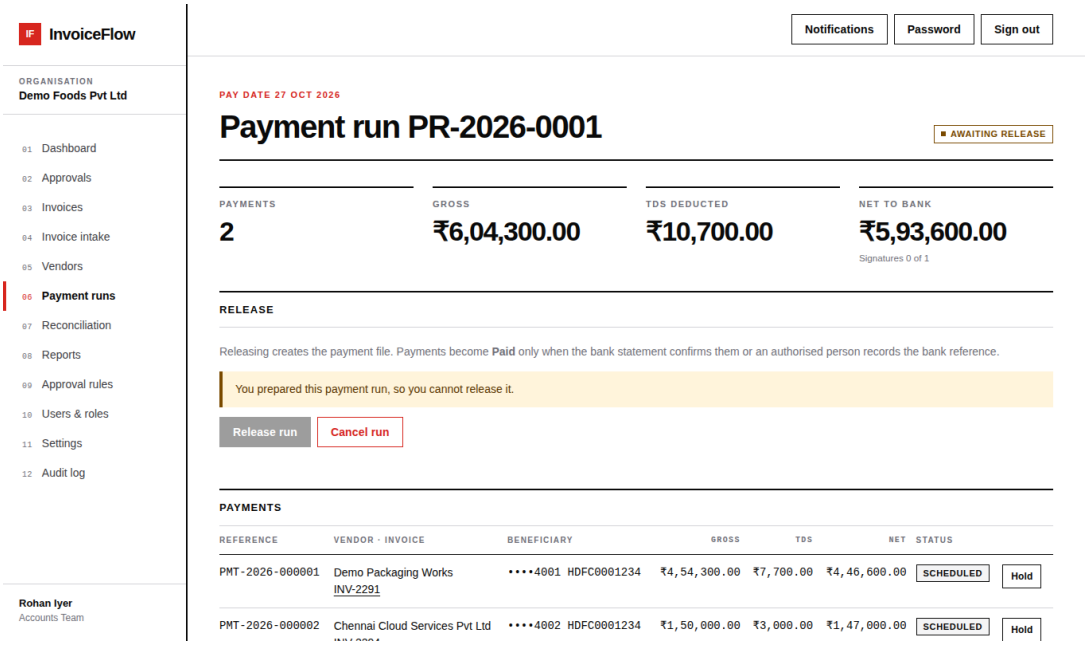


Figure A.5: Draft payment run: the preparer is told that someone else must release it

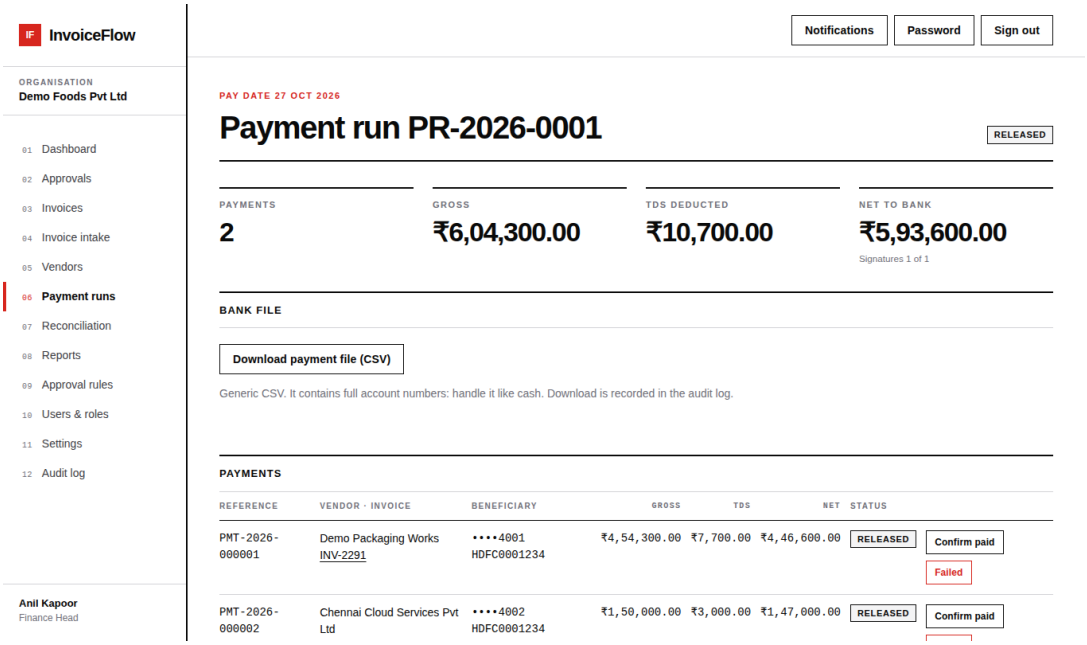


Figure A.6: Released payment run with bank-file download; payments are Released, not Paid

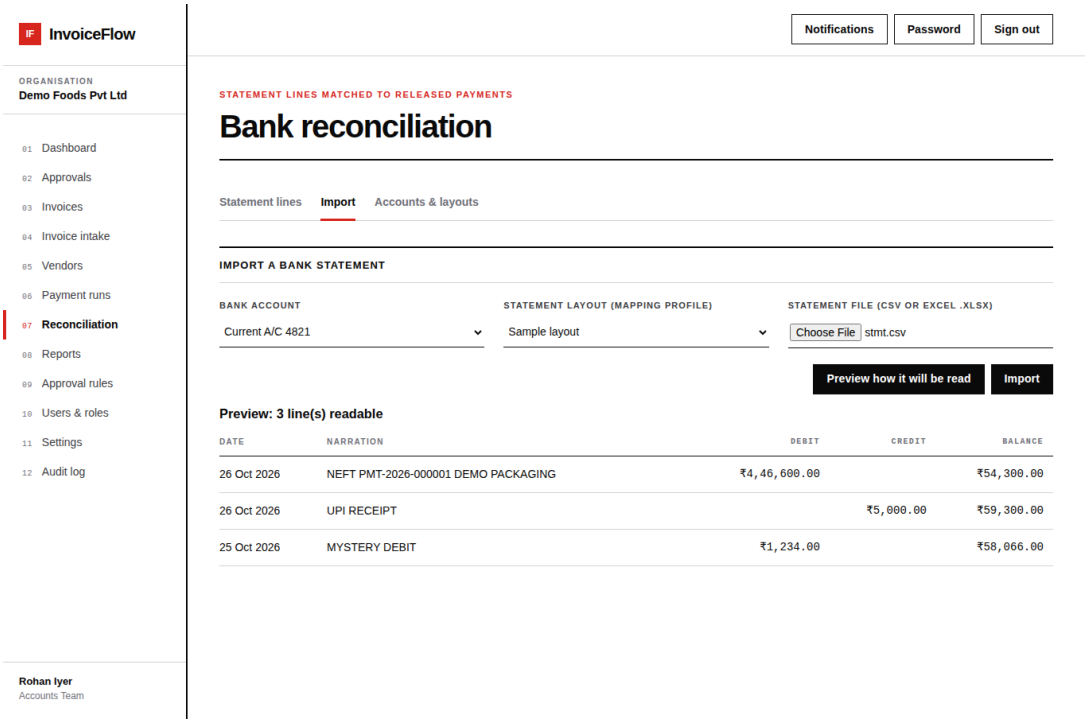


Figure A.7: Bank statement import preview with the selected layout and parsed lines

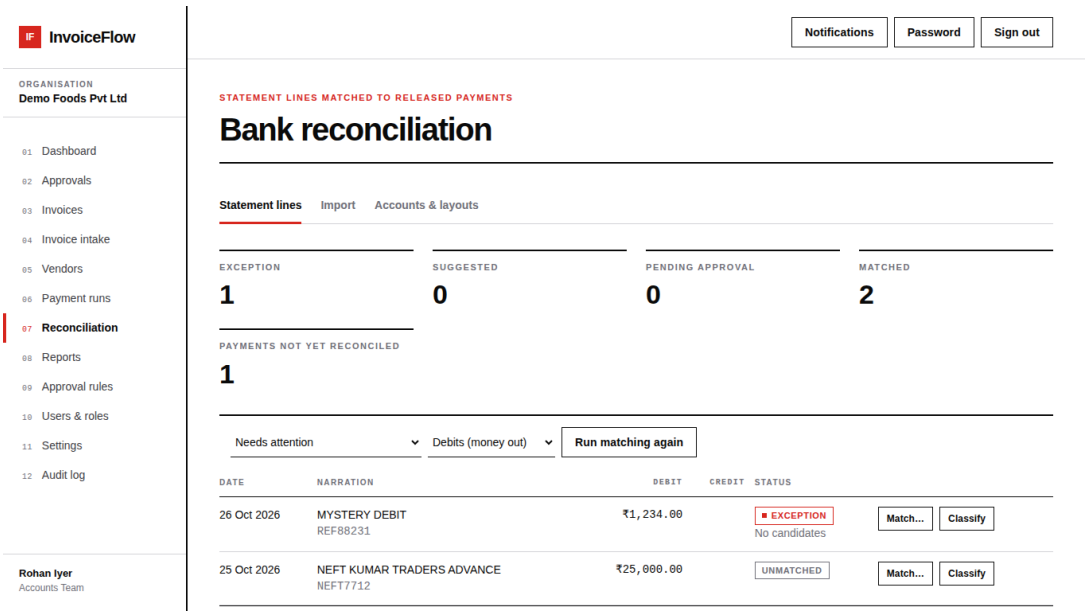


Figure A.8: Statement lines needing attention: an exception and an unmatched debit

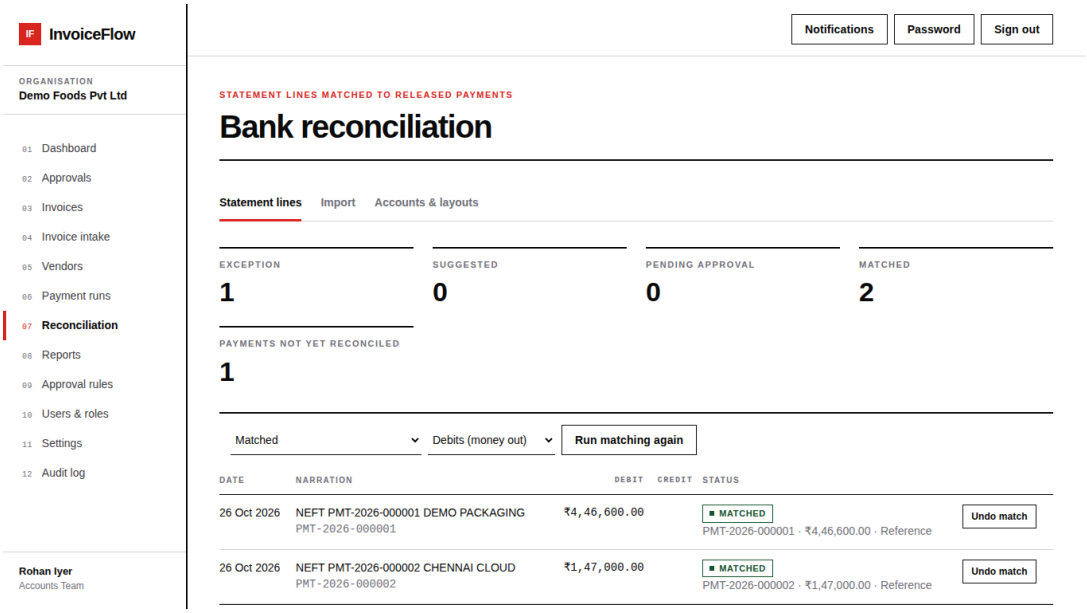


Figure A.9: Statement lines matched to payments by reference

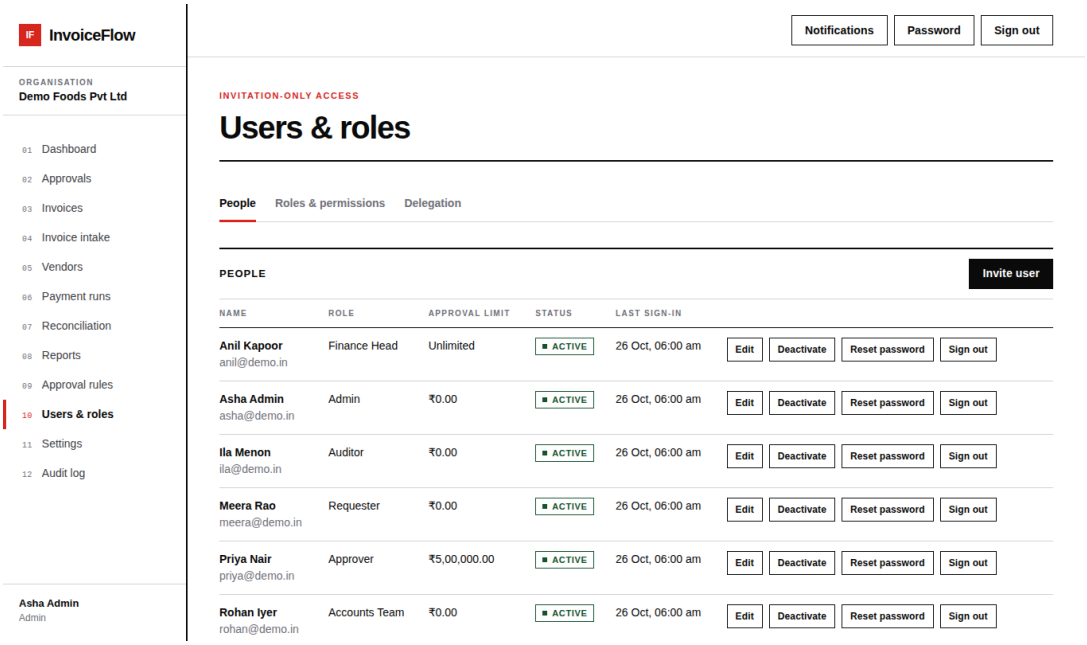


Figure A.10: Users and roles: people with approval limits

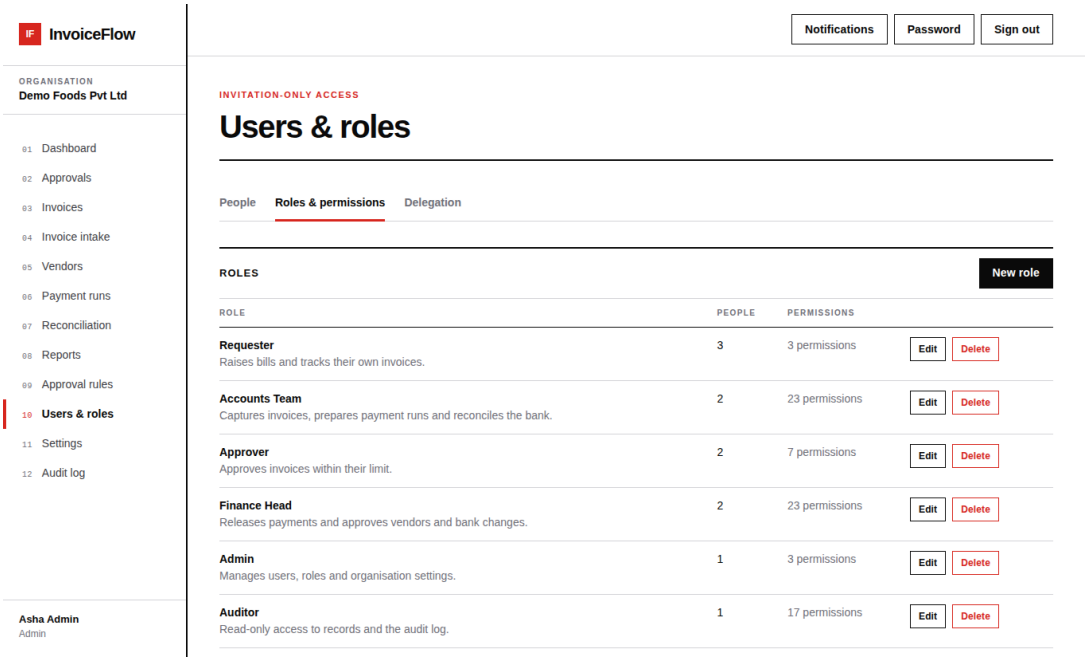


Figure A.11: Roles and permissions with editable role templates

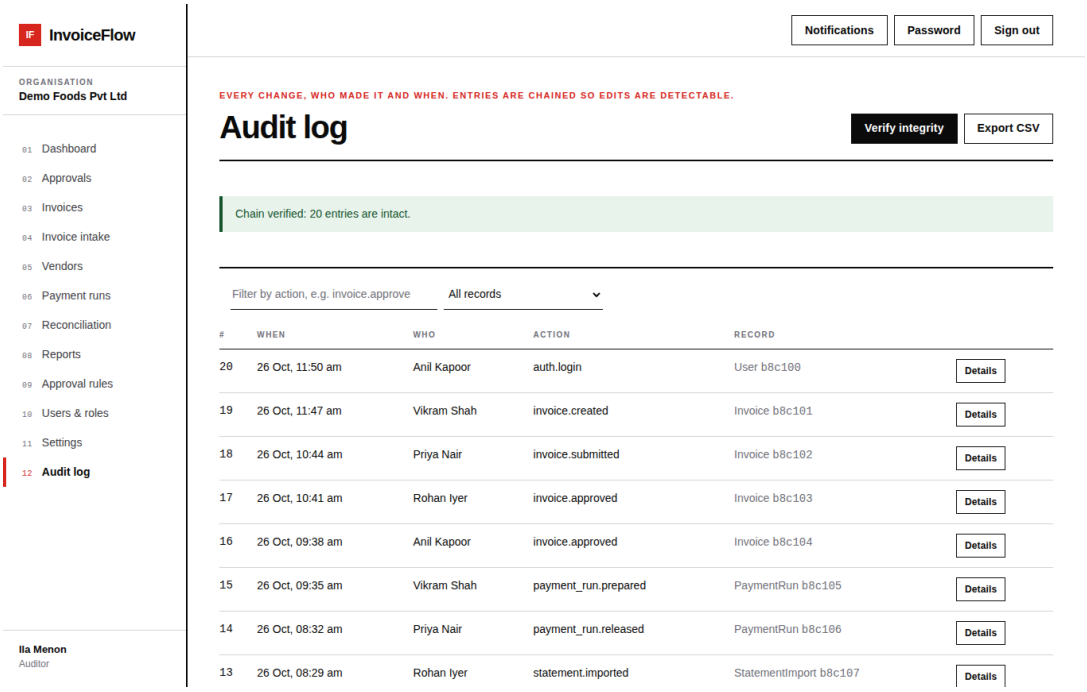


Figure A.12: Audit log with the chain-verified confirmation

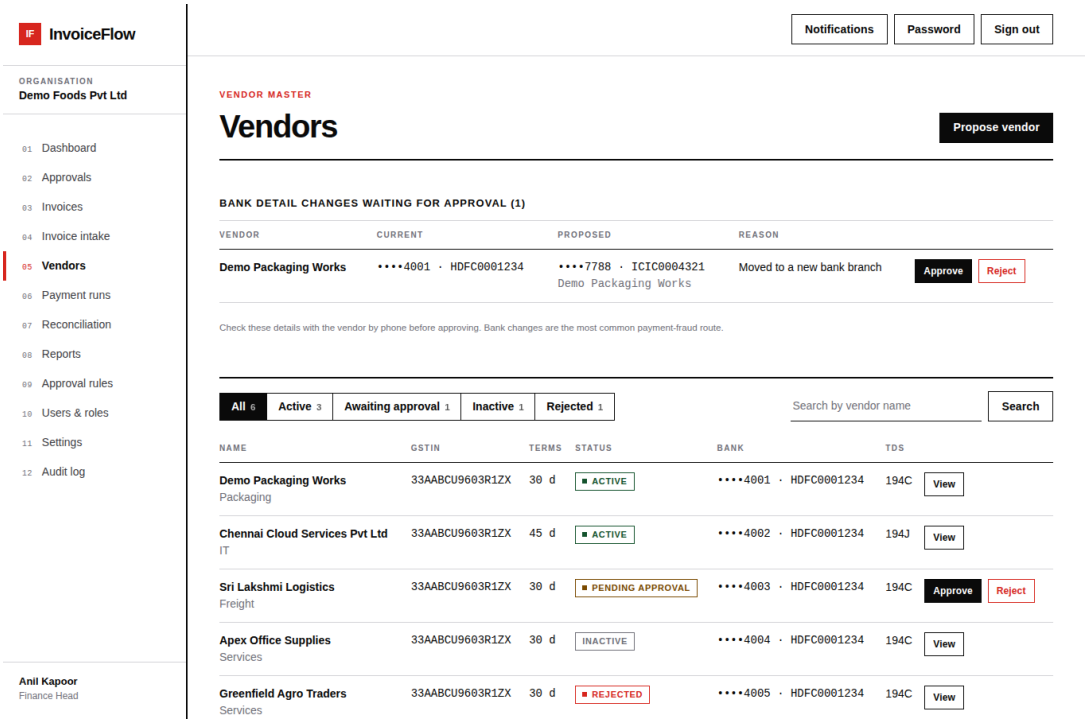


Figure A.13: Vendors: status filters with counts and the bank-change approval queue

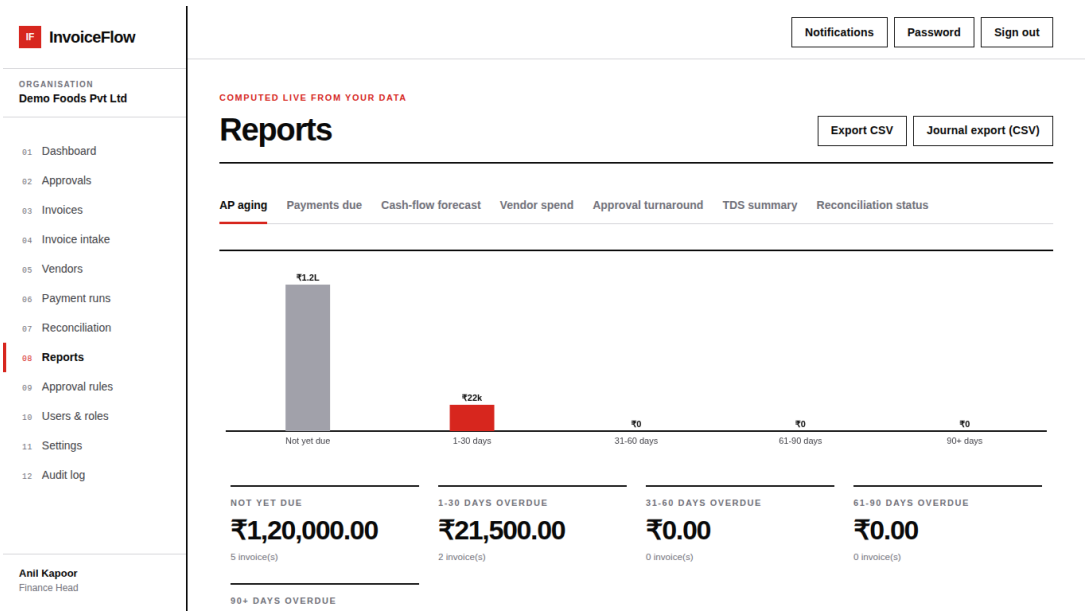


Figure A.14: Reports: AP aging chart, overdue buckets in red

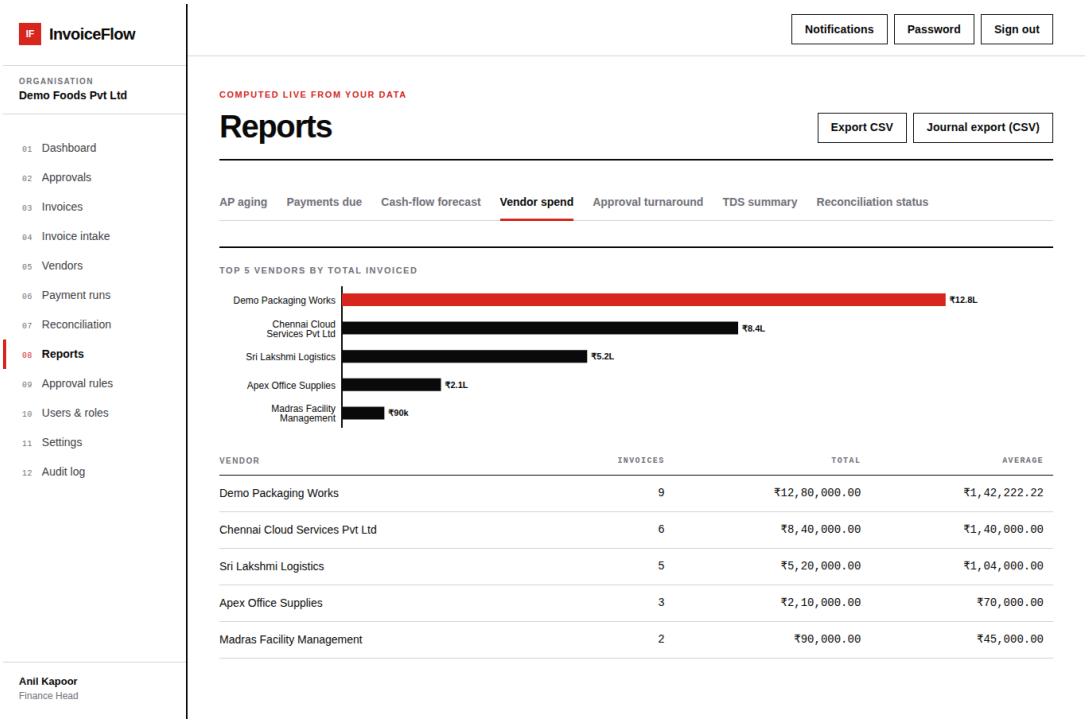


Figure A.15: Reports: vendor spend ranking

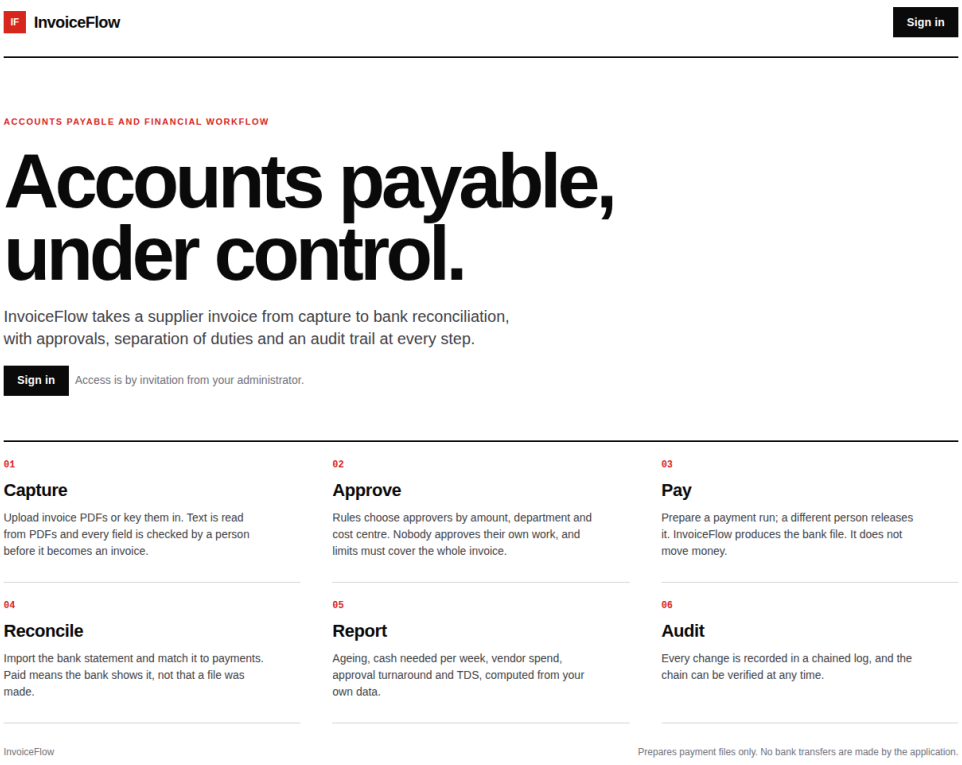


Figure A.16: Public landing page shown before sign-in (static content, no sample data)

REFERENCES

Citations in the text are numbered in square brackets. References [1]–[10], [12] and [20] are published works or standards; the remainder are official documentation or guidance. Page ranges and dates should be re-checked against the originals before submission.

[1] M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA, USA: Addison-Wesley, 2002.

[2] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Dept. of Information and Computer Science, Univ. of California, Irvine, CA, USA, 2000.

[3] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, May 2015.

[4] D. D. Clark and D. R. Wilson, "A Comparison of Commercial and Military Computer Security Policies," in Proc. IEEE Symposium on Security and Privacy, 1987, pp. 184–194.

[5] J. H. Saltzer and M. D. Schroeder, "The Protection of Information in Computer Systems," Proc. IEEE, vol. 63, no. 9, pp. 1278–1308, Sep. 1975.

[6] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-Based Access Control Models," IEEE Computer, vol. 29, no. 2, pp. 38–47, Feb. 1996.

[7] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST Standard for Role-Based Access Control," ACM Trans. Information and System Security, vol. 4, no. 3, pp. 224–274, Aug. 2001.

[8] B. Schneier and J. Kelsey, "Secure Audit Logs to Support Computer Forensics," ACM Trans. Information and System Security, vol. 2, no. 2, pp. 159–176, May 1999.

[9] N. Provos and D. Mazières, "A Future-Adaptable Password Scheme," in Proc. USENIX Annual Technical Conference (FREENIX Track), 1999.

[10] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," NIST Special Publication 800-38D, Nov. 2007.

[11] MongoDB, Inc., "Transactions," MongoDB Manual. [Online]. Available: https://www.mongodb.com/docs/manual/core/transactions/

[12] Y. Shafranovich, "Common Format and MIME Type for Comma-Separated Values (CSV) Files," RFC 4180, Internet Engineering Task Force, Oct. 2005.

[13] OWASP Foundation, "CSV Injection." [Online]. Available: https://owasp.org/www-community/attacks/CSV_Injection

[14] OWASP Foundation, "Application Security Verification Standard (ASVS)." [Online]. Available: https://owasp.org/www-project-application-security-verification-standard/

[15] Meta Platforms, Inc., "React Documentation." [Online]. Available: https://react.dev

[16] Express.js, "Express – Node.js web application framework." [Online]. Available: https://expressjs.com

[17] Mongoose, "Mongoose Documentation." [Online]. Available: https://mongoosejs.com/docs/

[18] Taskforce.sh Inc., "BullMQ Documentation." [Online]. Available: https://docs.bullmq.io

[19] Docker, Inc., "Docker Compose Documentation." [Online]. Available: https://docs.docker.com/compose/

[20] ISO, "ISO 20022 – Financial services: Universal financial industry message scheme." [Online]. Available: https://www.iso20022.org